



微信搜一搜

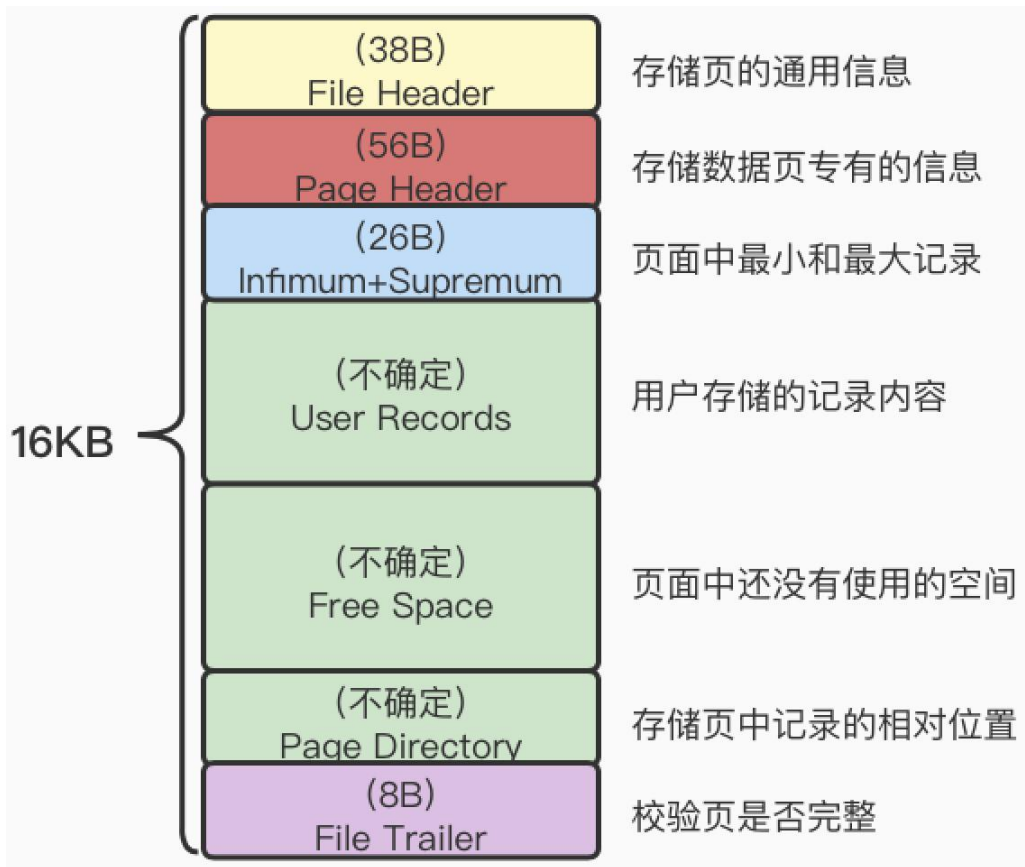
Q 爪哇繆斯

MySQL InnoDB存储引擎

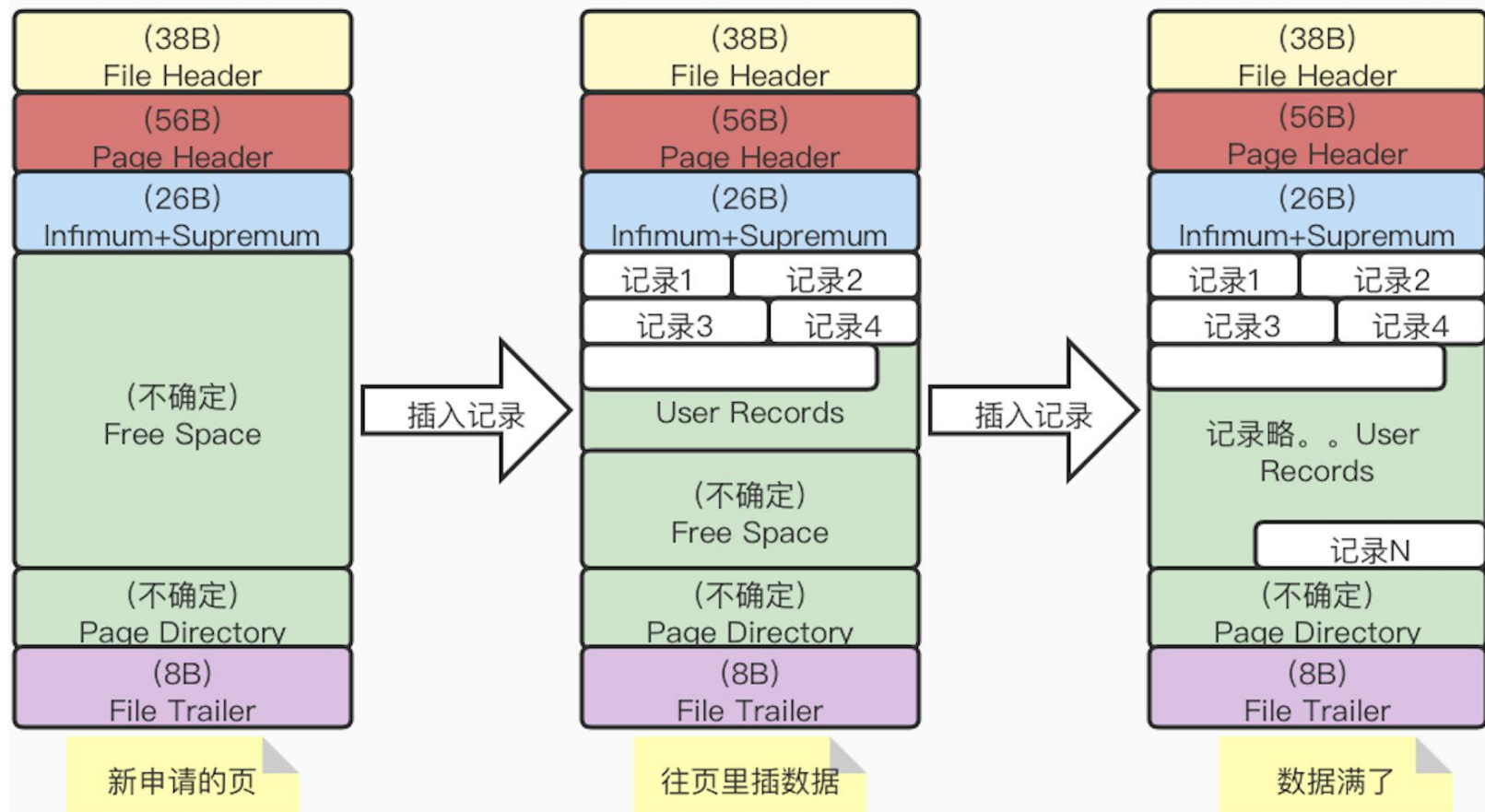
| Page——页

InnoDB数据页及其结构

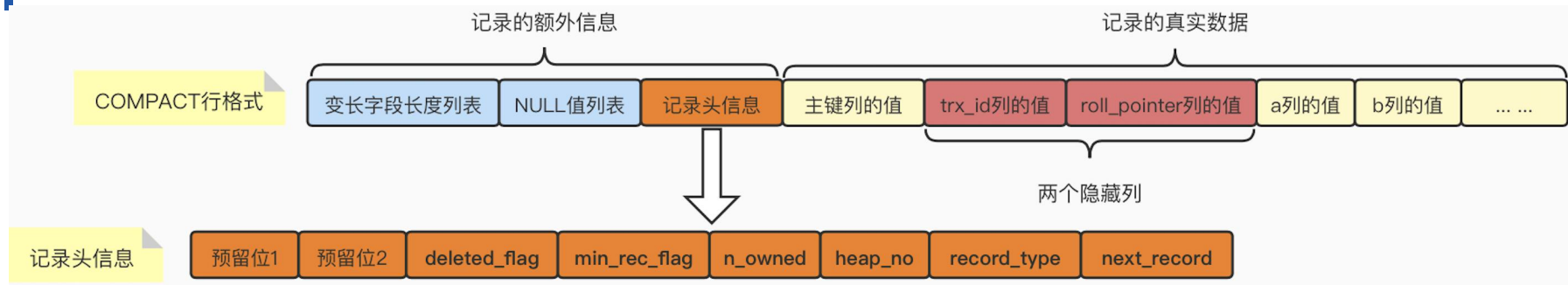
- 为了避免一条一条读取磁盘数据，InnoDB采取页的方式，作为磁盘和内存之间交互的基本单位。一个页的大小一般是16KB。
- InnoDB为了不同的目的而设计了多种不同类型的页。比如：存放表空间头部信息的页、存放undo日志信息的页等等。我们把存放表中数据记录的页，称为索引页or数据页。



往数据页中存储数据（也叫“记录”）



记录的头信息



- **deleted_flag**: 逻辑删除标记 (0:未删除 1:已删除)
- **min_rec_flag**: B+树中每层非叶子节点中的最小的目录项记录, 都会添加该标记。
- **n_owned**: 一个页面被分若干组后, “带头大哥”用于保存组中所有的记录条数。
- **heap_no**: 表示当前记录在页面堆中的相对位置。
- **record_type**: 表示当前的记录类型。
 - ① 0: 普通记录
 - ② 1: B+树非叶子节点的目录项记录
 - ③ 2: 表示Infimum记录
 - ④ 3: 表示Supremum记录
- **next_record**: 表示下一条记录的相对位置, 也就是链表。这个属性非常重要。它表示从当前记录的真实数据到下一条记录的真实数据的距离。

记录的头信息

- 当我们了解到了记录的数据结构之后，我们以向tb_student表中插入4条数据为例，来看一下记录之间数据结构存储情况是怎么样的。
- 下面我们创建一张tb_student表，表中有4个字段，分别是id、number、name和age，然后插入表中的4条数据如下图所示：

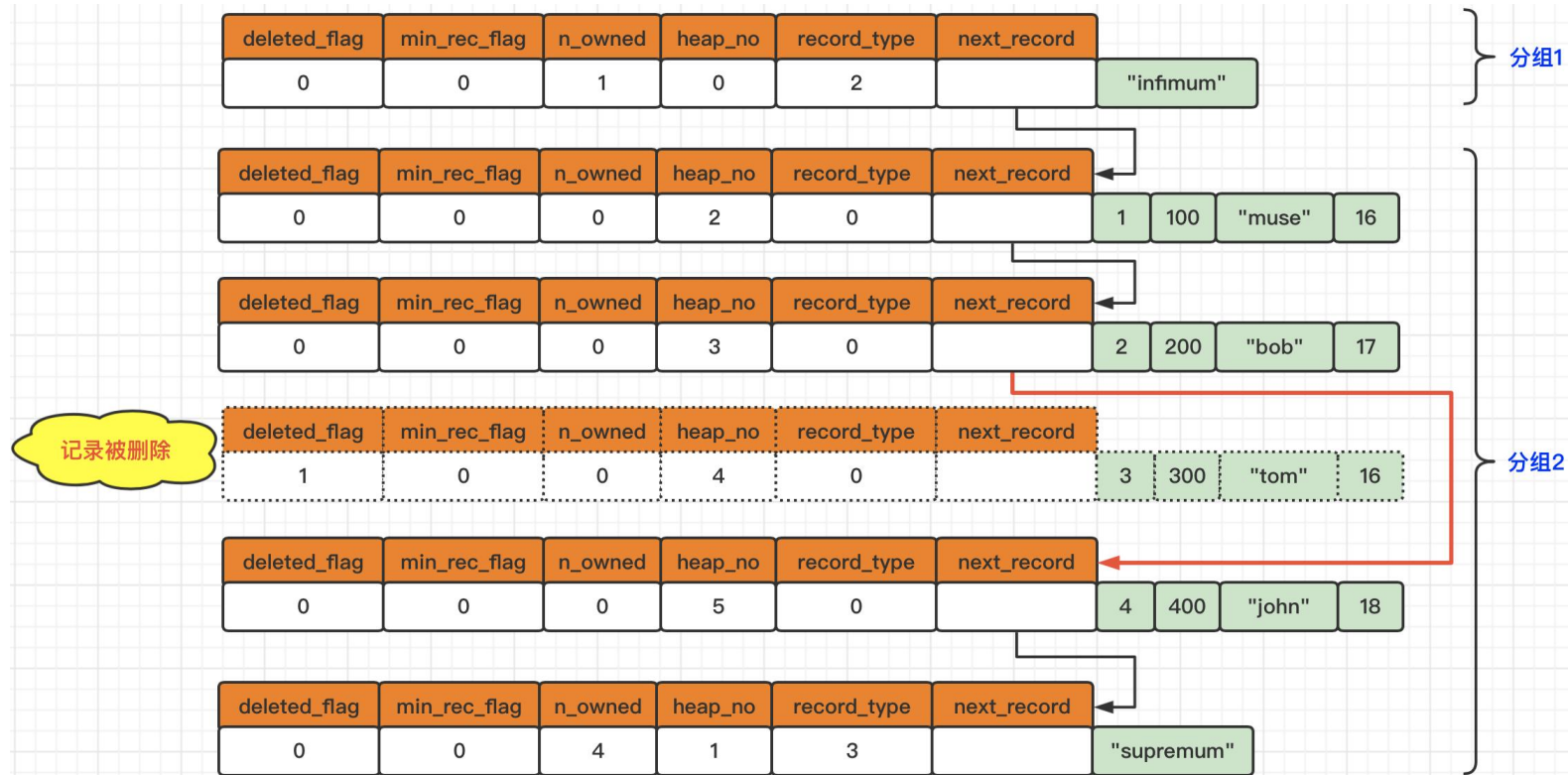
tb_student			
id	number	name	age
1	100	muse	16
2	200	bob	17
3	300	tom	16
4	400	john	18

【学生信息表 tb_student】

- id: int类型的主键
- number: int类型的学生学号
- name: varchar(30) 学生姓名
- age: int类型的学生年龄

删除记录操作对next_record的影响

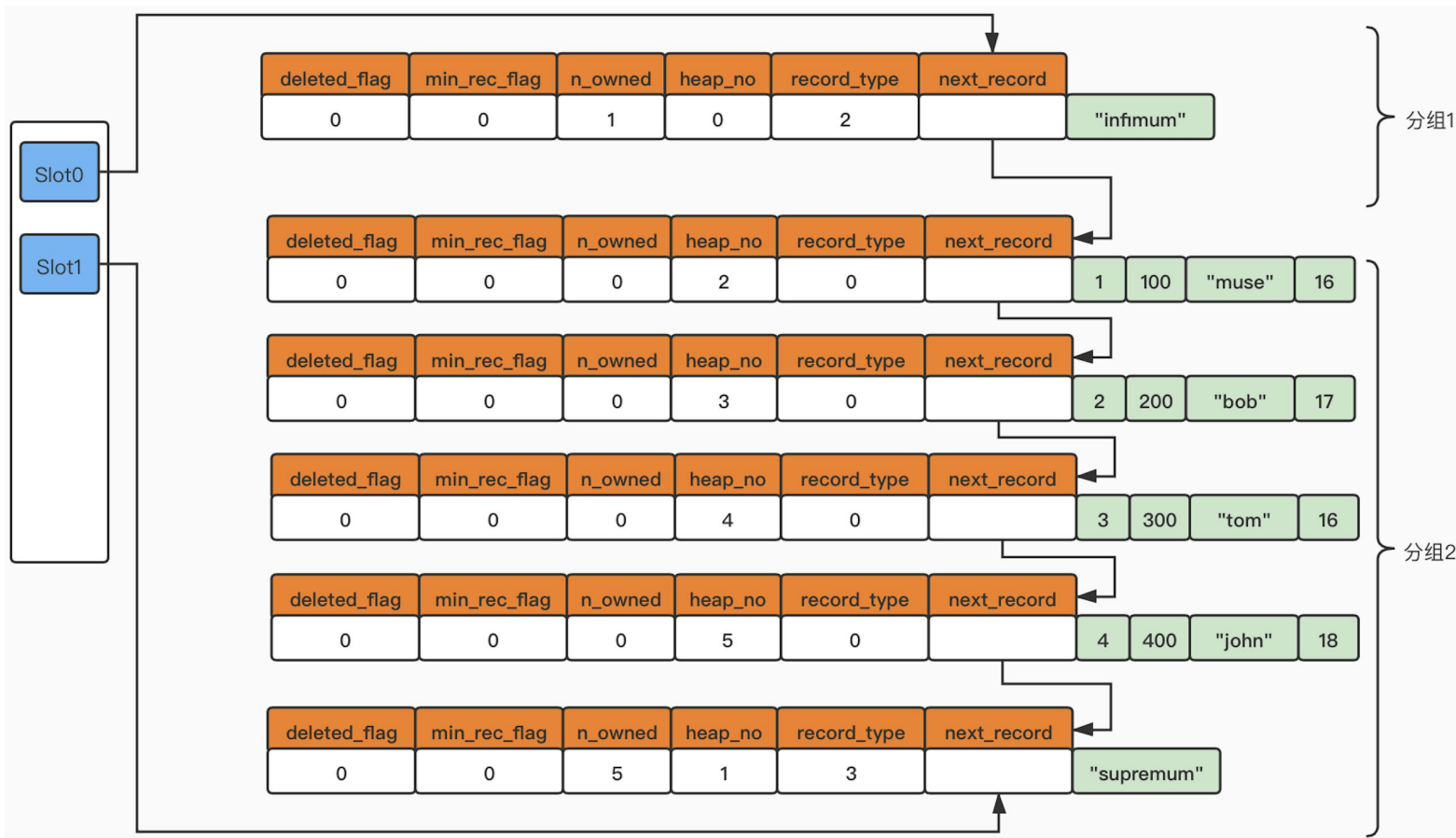
- 可以看出记录是按照主键从小到大的顺序形成了一个单向链表。记录被删除对next_record的影响，如下图所示：



Page Directory 概述

- 记录在页中是按照主键值从小到大的顺序串联成为一个单向链表，因此查询也只能以头节点开始逐一向后查询，但是如果数据量很大，那么性能就无法保证了。针对这个问题，InnoDB采取了图书目录的解决方案，即：Page Directory。
- 分组规则如下所示：
 - ① 对于Infimum记录所在的分组只能有1条记录。
 - ② 对于Supremum记录所在的分组只能在1~8条记录之间。
 - ③ 剩下的其他记录所在的分组只能在4~8条记录之间。
- 分组步骤如下：
 - ① 初始情况下，一个数据页中只有Infimum记录和Supremum记录这两条，所以分为两个组。
 - ② 之后每当插入一条记录时，都会从页目录中找到对应记录的主键值比待插入记录的主键值大，并且差值最小的槽，然后把该槽对应的n_owned加1。
 - ③ 当一个组中的记录数等于8时，当再插入一条记录的时候，会将组中的记录拆分成两个组（一个组中4条记录，另一个组中5条记录）。并在拆分过程中，会在Page Directory中新增一个槽，并记录这个新增分组中最大的那条记录的偏移量。

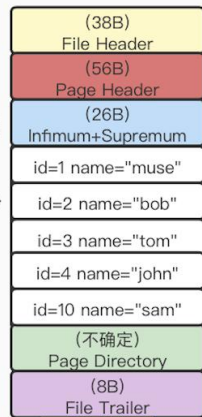
Page Directory——记录在页中的展现



页满后的插入操作

```
INSERT INTO tb_student VALUES (3, 300, 'tom', 16);  
INSERT INTO tb_student VALUES (1, 100, 'muse', 16);  
INSERT INTO tb_student VALUES (4, 400, 'john', 18);  
INSERT INTO tb_student VALUES (2, 200, 'bob', 17);  
INSERT INTO tb_student VALUES (10, 1000, 'sam', 19);
```

乱序插入，底层
依然有序存储



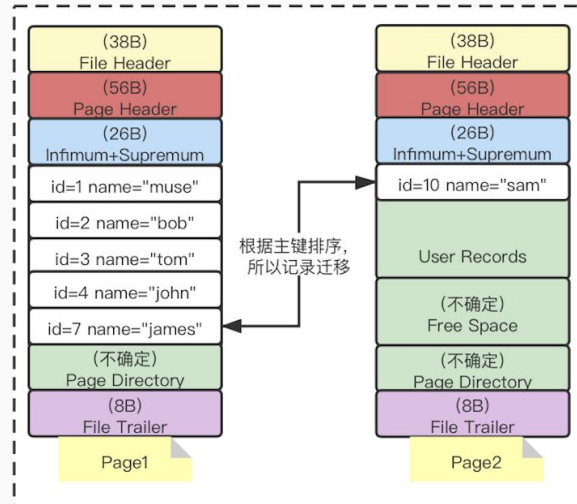
Page1

记录存满了，
开辟新页



Page2

插入id=7的
James



数据主键和业务主键
的区别是什么？

```
INSERT INTO tb_student VALUES (3, 300, 'tom', 16);  
INSERT INTO tb_student VALUES (1, 100, 'muse', 16);  
INSERT INTO tb_student VALUES (4, 400, 'john', 18);  
INSERT INTO tb_student VALUES (2, 200, 'bob', 17);
```

17 • select * from tb_student;

100%

50:15

Result Grid



Filter Rows:



Search

	id	number	name	age
▶	1	100	muse	16
	2	200	bob	17
	3	300	tom	16
	4	400	john	18

思考问题

我们已经介绍完了MySQL InnoDB记录存储的底层数据结构。

那么问题来了：

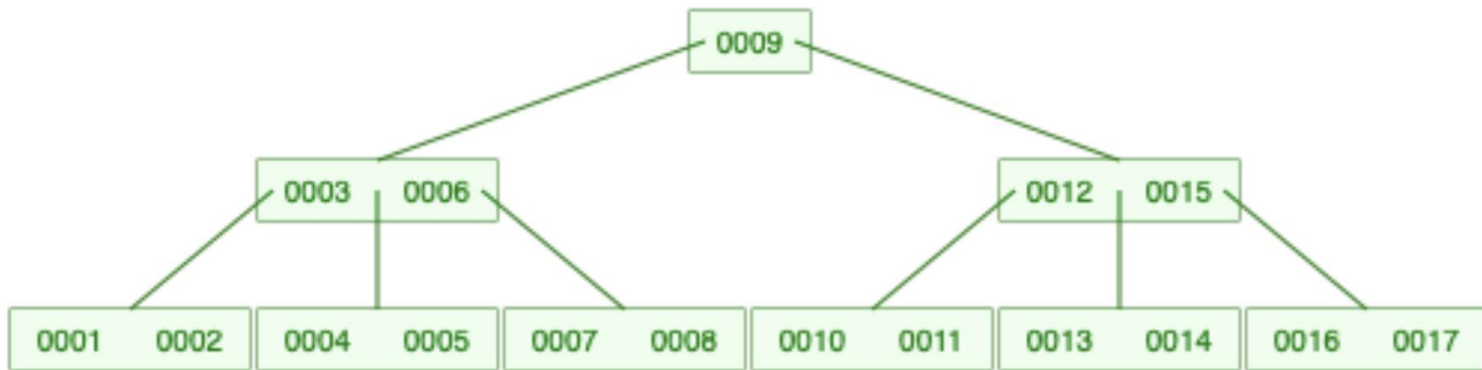
如果我们要查询某一条记录，查询过程是怎样的？



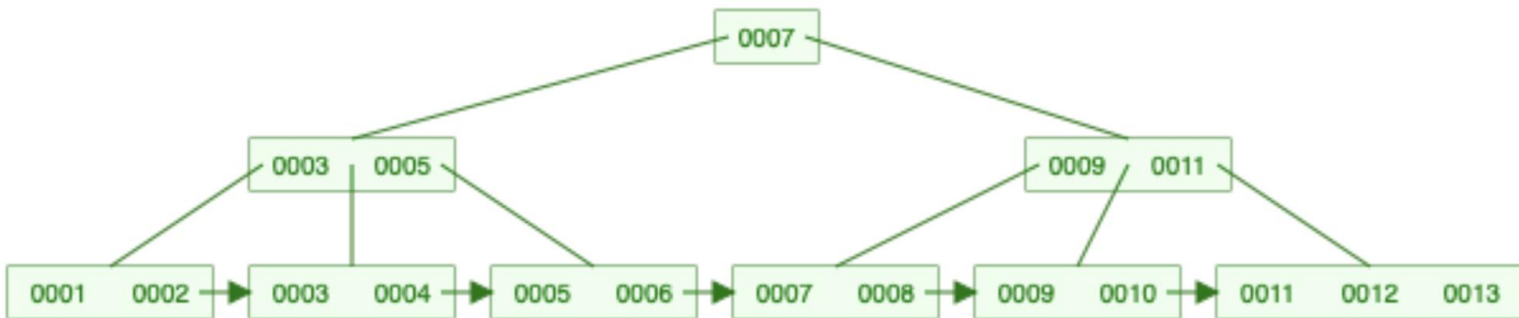
B 树 & B+ 树

B树和B+树的区别（阶数=5）

➤ B树



➤ B+树



B+树的特点

【B树&B+树的插入和删除图文详解】

<https://www.cnblogs.com/nullzx/p/8729425.html>

【与B树相同点】

- 一个节点可以**存储多个元素**。
- 与B树一样，叶子节点是**有序的**。
- 每个节点中的元素，也都按照**从小到大**的顺序排列，即：**左小右大**。
- **所有叶子节点都位于同一层**，或者说根节点到每个叶子节点的高度都相同。

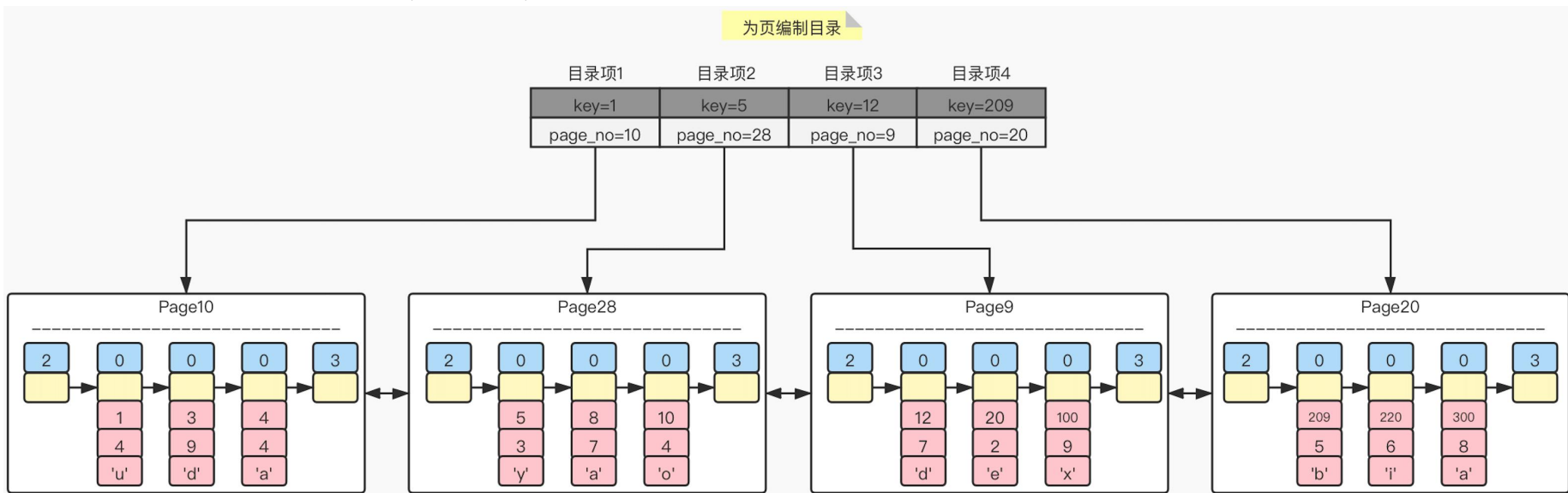
【与B树不同点】

- B+树的叶子节点是**有单向指针**的，其中：MySQL中采用的是双向指针。
- B+树的非叶子节点的元素是与叶子节点有**冗余的**。

Index——索引

如何加快记录的查询

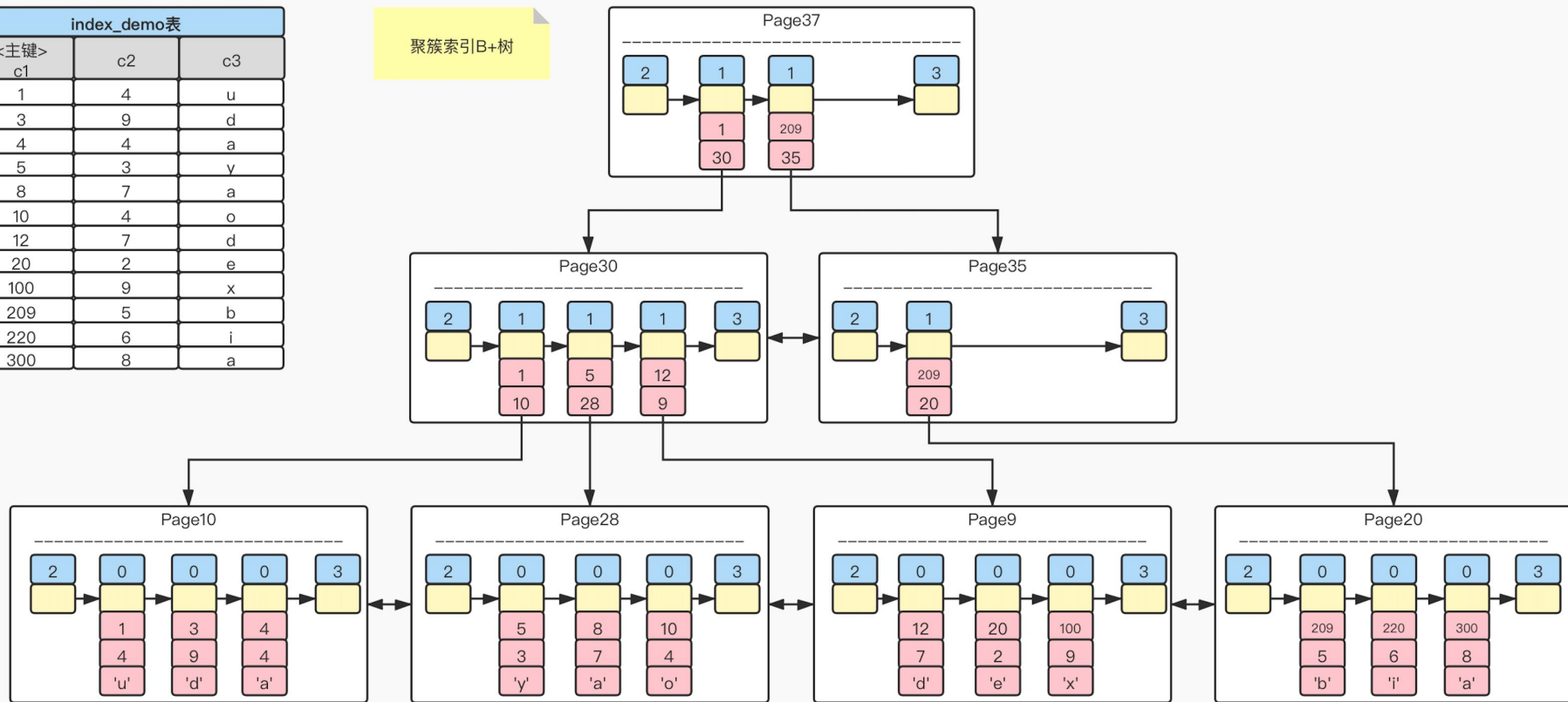
- 当记录越来越多，创建的页也会越来越多，如果仅通过链表方式遍历查询，性能会出现很大问题。如何解决呢？ 采用B+树结构，即：叶子节点里存储完整的数据（数据页），非叶子节点存储主键索引（索引页）



聚簇索引——主键

聚簇索引B+树

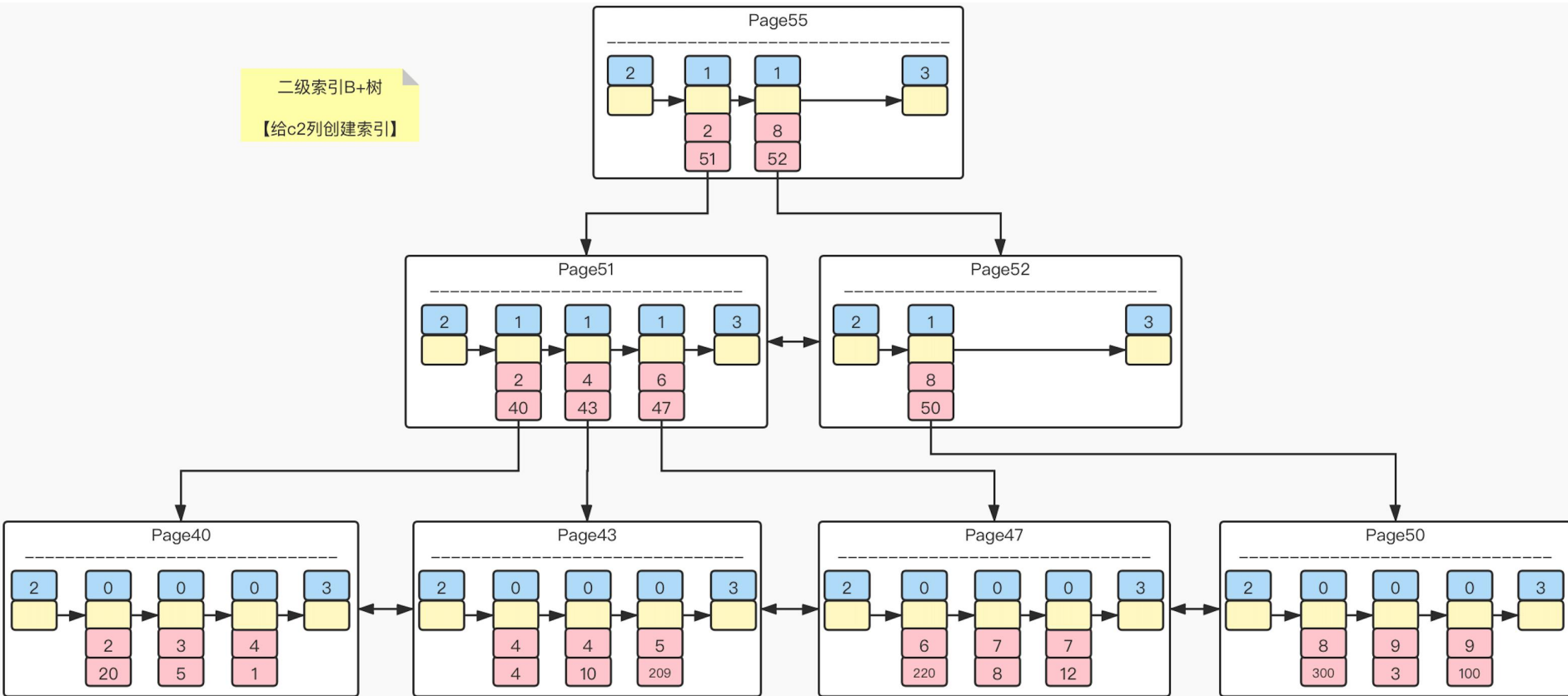
index_demo表		
<主键> c1	c2	c3
1	4	u
3	9	d
4	4	a
5	3	y
8	7	a
10	4	o
12	7	d
20	2	e
100	9	x
209	5	b
220	6	i
300	8	a



二级索引——非主键

二级索引B+树

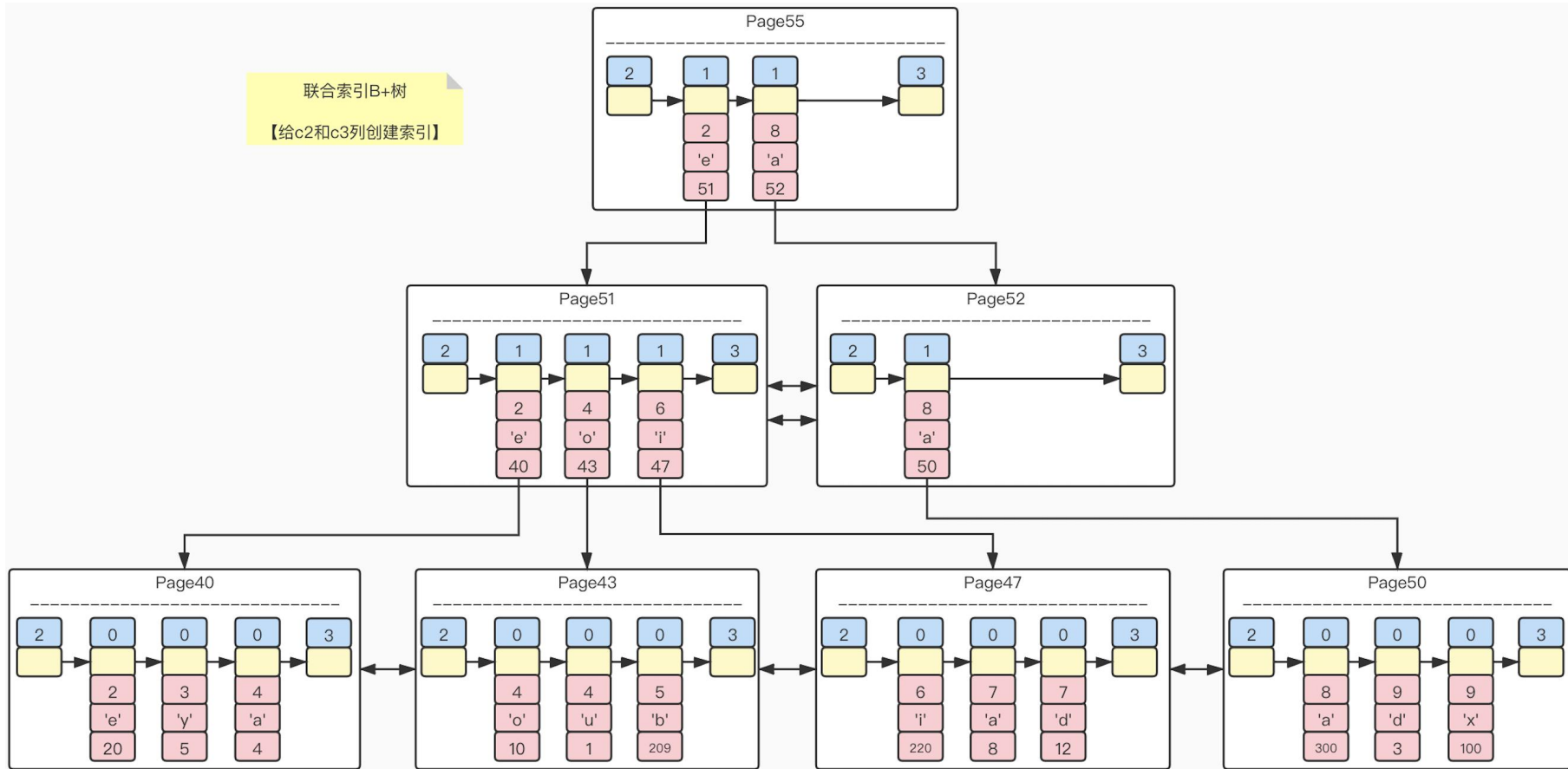
【给c2列创建索引】



联合索引——多列

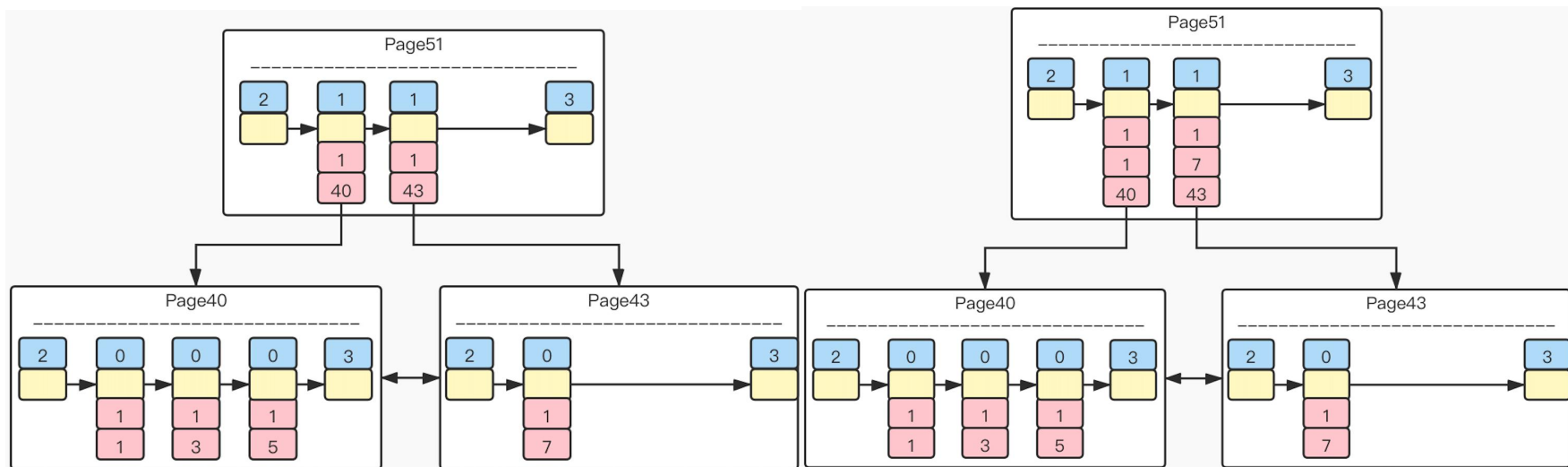
联合索引|B+树

【给c2和c3列创建索引】



目录项记录的唯一性

- 为了让新插入的记录能找到自己在哪个页中，就需要保证B+树同一层内节点的目录项记录除页号字段外是唯一的。所以二级索引的内节点的目录项记录的内容实际是有3部分构成的：索引列的值(c2)+主键值(c1)+页号(pageNo)。这样，如果c2列的值相同，那么可以接着比较主键值。所以，归根结底，我们可以认为，为c2列建立的二级索引其实相当于为 (c2, c1) 列建立了一个联合索引。

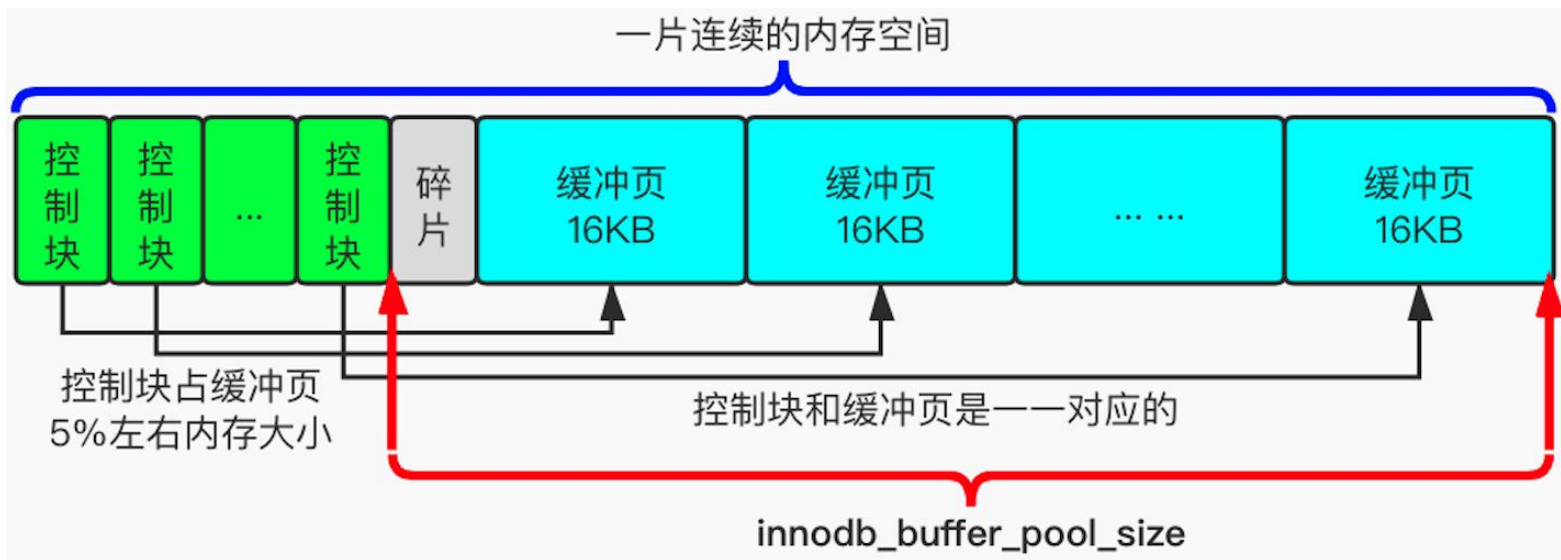




Buffer Pool——缓冲池

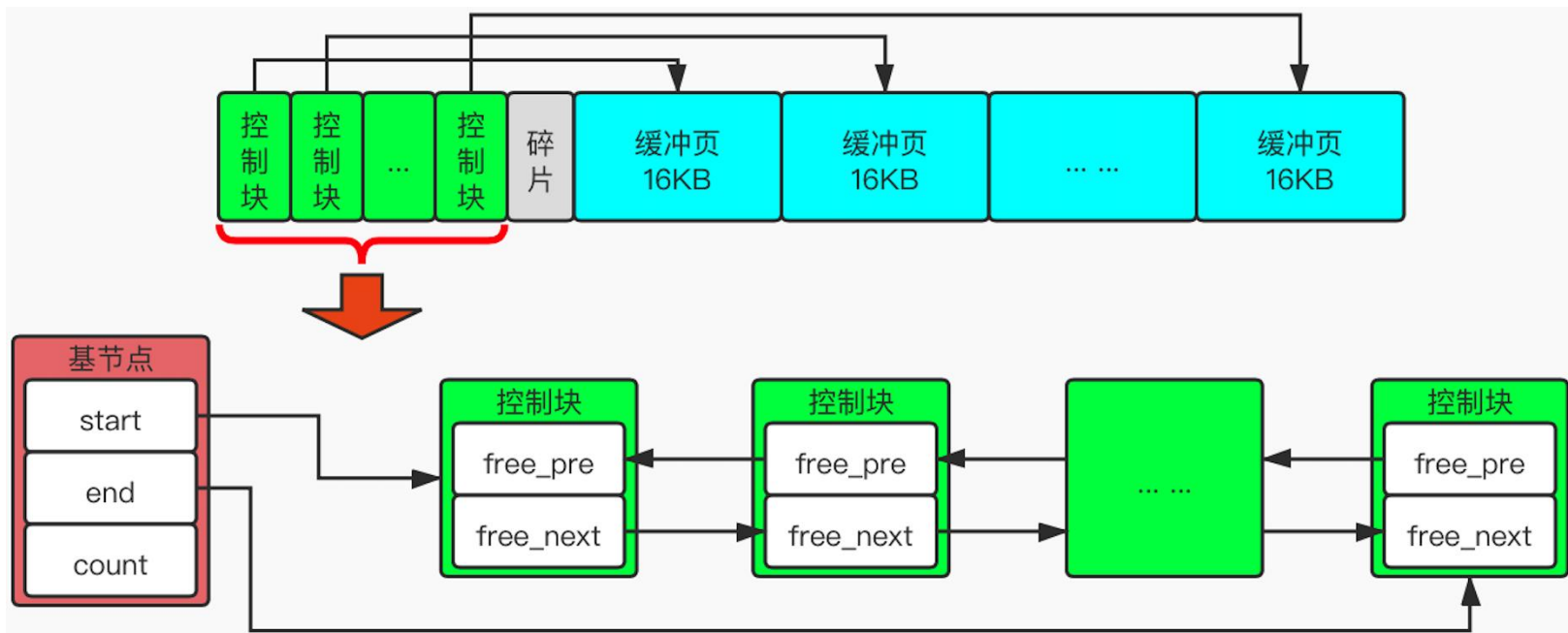
缓冲池概述

- 为了缓存磁盘中的页，MySQL服务器启动时就向操作系统申请了一片连续的内存空间，他们给这片内存起名为Buffer Pool（缓冲池）。默认Buffer Pool只有128M，可以在启动服务器的时候配置`innodb_buffer_pool_size`（单位为字节）启动项来设置自定义缓冲池大小。Buffer Pool对应的一片连续的内存被划分为若干个页面，默认也是16KB，该页面称为缓冲页。为了更好的管理Buffer Pool中的这些缓冲页，InnoDB为每个缓冲页都创建了控制块，它与缓冲页是一一对应的。



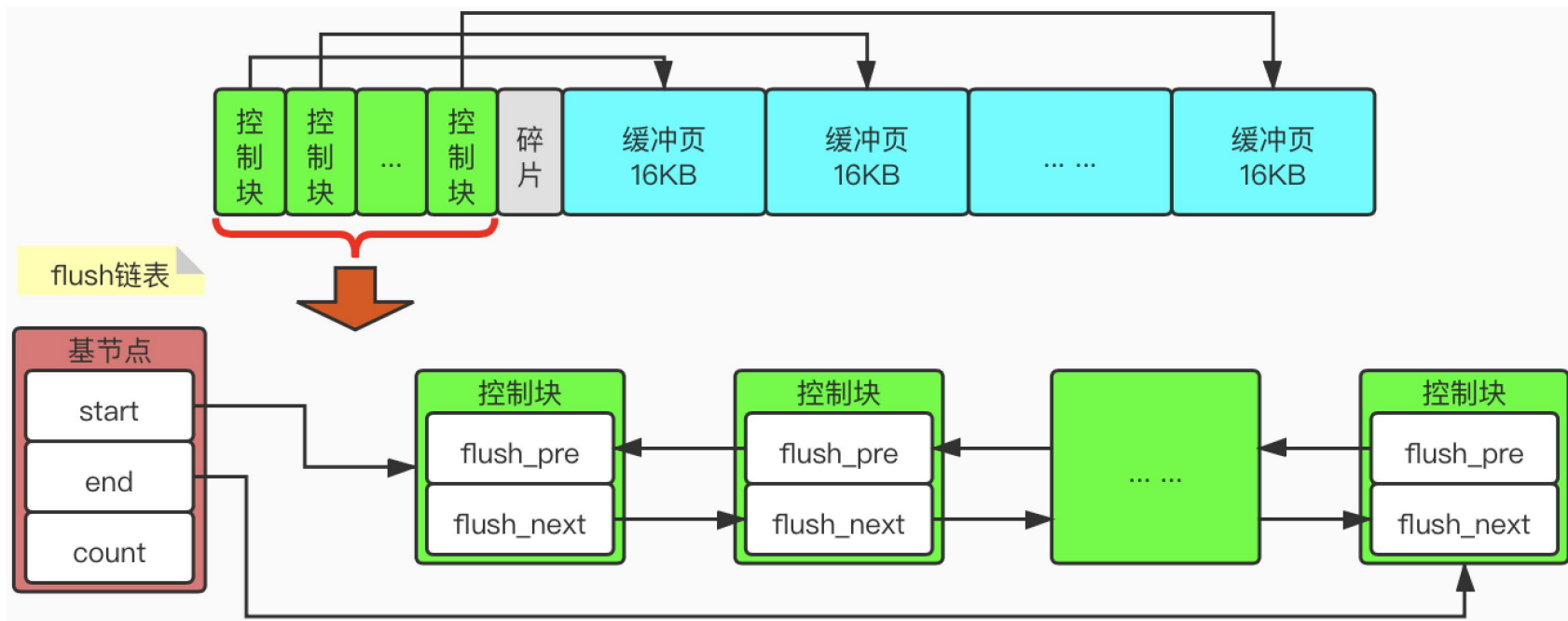
Free 链表

- Buffer Pool的初始化过程中，是先向操作系统申请连续的内存空间，然后把它划分成若干个【控制块&缓冲页】对儿。当插入数据的时候，为了能够知道哪些缓冲页是空闲且可分配的，MySQL把所有空闲的缓冲页对应的控制块作为一个节点放到一个链表中，这个链表便称之为free链表。



Flush 链表

- 如果我们修改了Buffer Pool中某个缓冲页的数据，那么它就与磁盘上的页不一致了，这样的缓冲页也被称之为脏页（dirty page）。为了性能问题，我们每次修改缓冲页后，并不着急立刻把修改刷新到磁盘上，而是将被修改过的缓冲页对应的控制块作为节点加入到这个链表中，该链表也被称为flush链表。



Flush链表刷新方式有如下两种：

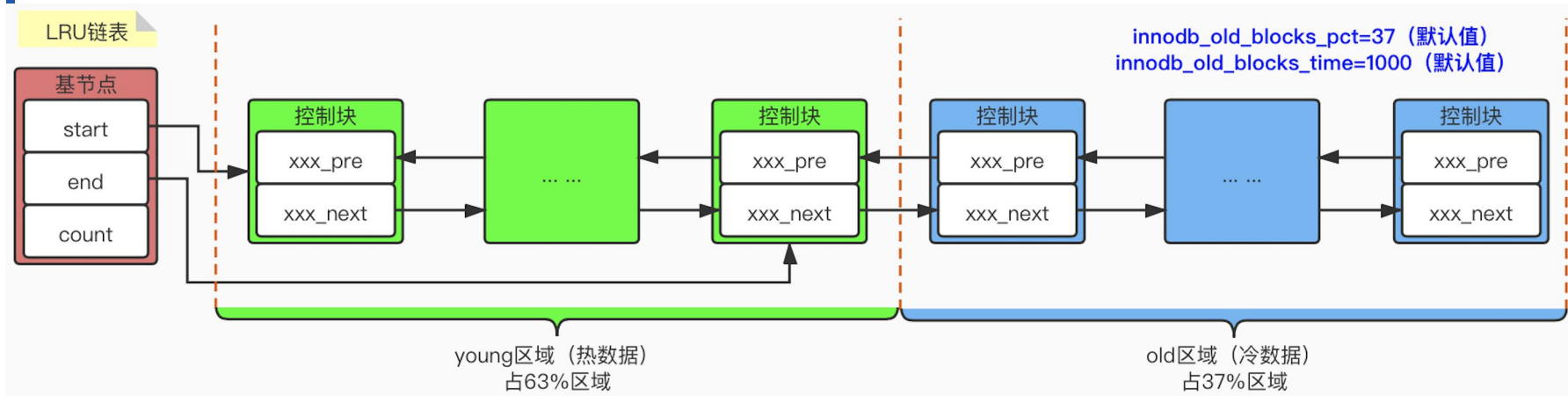
【1】从flush链表中刷新一部分页面到磁盘

- 后台线程也会定时从flush链表中刷新一部分页面到磁盘，刷新的速率取决于当时系统是否繁忙。——即： **BUF_FLUSH_LIST**
- 有时后台线程刷新脏页的进度比较慢，导致用户准备加载一个磁盘页到Buffer Pool中时没有可用的缓冲页。此时，就会尝试查看LRU链表尾部，看是否存在可以直接释放掉的未修改缓冲页。如果没有，则不得不将LRU链表尾部的一个脏页同步刷新到磁盘（与磁盘交互是很慢的，这会降低处理用户请求的速度）。——即： **BUF_FLUSH_SINGLE_PAGE**

【2】从LRU链表的冷数据中刷新一部分页面到磁盘

- 后台线程会定时从LRU链表的尾部开始扫描一些页面，扫描的页面数量可以通过系统变量 `innodb_lru_scan_depth` 来指定，如果在LRU链表中发现脏页，则把它们刷新到磁盘。——即： **BUF_FLUSH_LRU**
- 控制块里会存储该缓冲页是否被修改的信息，所以在扫描LRU链表时，可以很轻松地获取到某个缓冲页是否是脏页的信息。

LRU 链表



- **线性预读**: 如果顺序访问**某个区** (extent, 一个区默认64个页) 的页面超过了 `innodb_read_ahead_threshold` (默认**56**) 的值, 就会触发一次异步读取**下一个区**中全部的页到Buffer Pool中的请求。
- **随机预读**: 如果开启了随机预读功能 (默认: `innodb_random_read_ahead=OFF`), 如果某个区 (extent) 有**13个连续的页面**都已经被加载到了Buffer Pool中, 无论这些页面是不是顺序读取的, 都会触发一次异步读取**本区**全部的页到Buffer Pool中的请求。

针对LRU链表方案缺点的优化

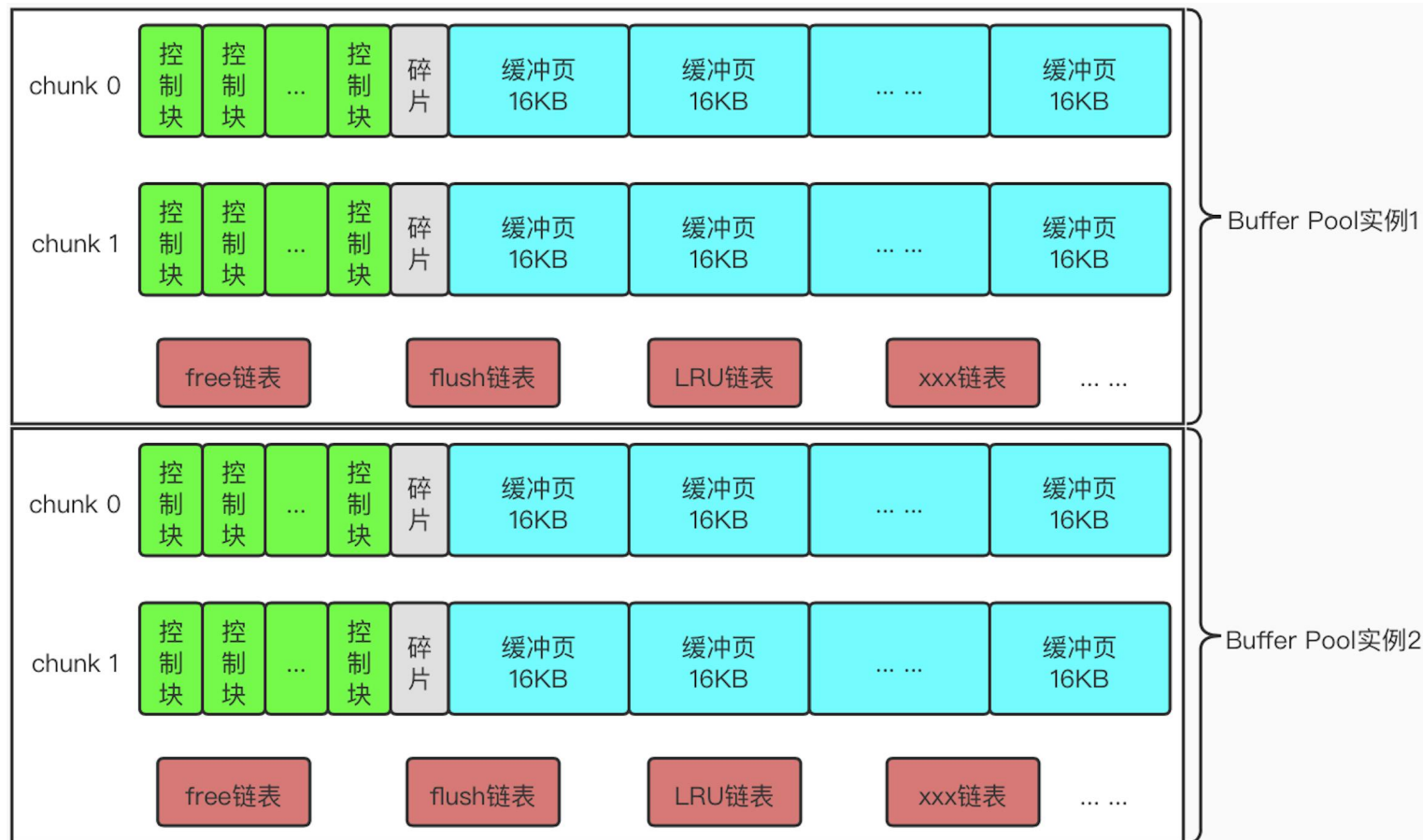
➤ 针对预读的优化

- ① InnoDB规定，当磁盘上的某个页在**初次加载**（只是加载，没有涉及读取）到Buffer Pool中的某个缓冲页时，该缓冲页对应的控制块会被放到old区域的头部。这样预读页就只会old区域，不会影响young区域中使用比较频繁的缓冲页。

➤ 针对全表扫描的优化

- ① 虽然首次加载放到的是old区域的头部，但是由于是全表扫描，会对加载的数据进行访问，那么第一次访问的时候，就会将该页放到young区域的头部。这样仍然会把那些使用频率比较高的页面给“排挤”下去。
- ② 那怎么办呢？由于全表扫描有一个特点，就是它对某个页的频繁访问且总耗时很短。所以，针对这种情况，InnoDB规定，在对某个处于old区域的缓冲页进行第一次访问时，就在它对应的控制块中记录下这个访问时间，**如果后续的访问时间与第一次访问的时间在某个时间间隔内（即：innodb_old_blocks_time，默认为1000，单位为ms），那么该页面就不会从old区域移动到young区域的头部，否则将它移动到young区域的头部。**

chunk和Buffer Pool实例



| redo 日志

什么是redo日志

➤ 如果我们只在内存的Buffer Pool中修改了页面，假设在事务提交后突然发生了某个故障，导致内存中的数据都失效了，那么这个已经提交的事务在数据库中所做的更改也就丢失了。针对这种问题，怎么处理呢？

➤ 【方案1】在事务提交时，把该事务修改的所有页面都刷新到磁盘，刷新成功了才提示事务提交成功

① 刷新一个完整的数据页太浪费了

虽然我们只修改了一条记录，但是会将这条记录所在的页（16KB）都刷新到磁盘上，会造成大量磁盘I/O的浪费。

② 随机I/O刷新起来比较慢

一个事务可能包含很多语句，即使是一条语句也可能修改许多页面，并且该事务修改的这些页面可能并不相邻。这就意味着将某个事务修改的Buffer Pool中的页面刷新到磁盘时，需要进行很多的随机I/O。而随机I/O要比顺序I/O慢，尤其是机械硬盘。

➤ 【方案2】在事务提交时，只需要把修改的内容记录一下就好了

例如：“将第0号表空间第100号页面中偏移量为1000处的值更新为2。”

redo 简单日志类型

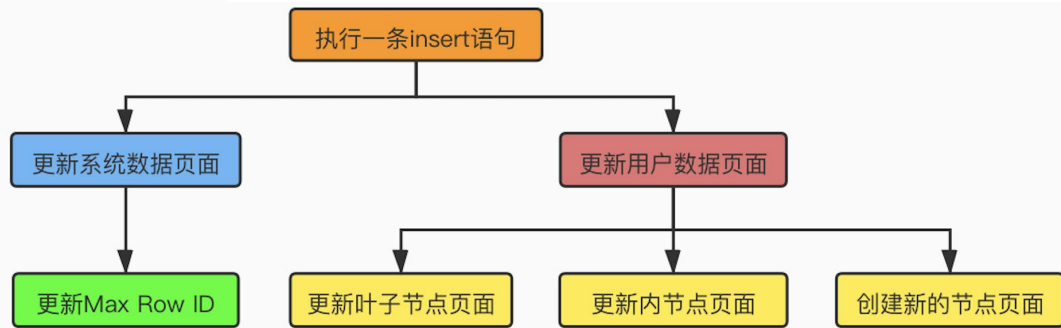
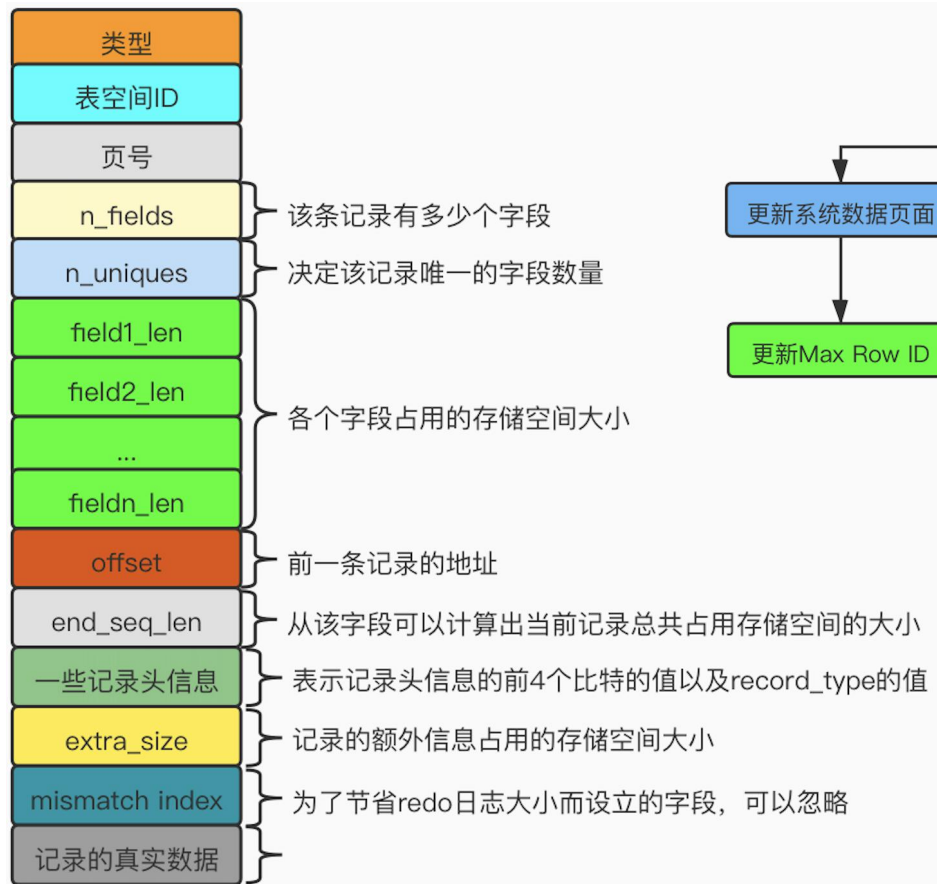
- 在对页面的修改是极其简单的情况下（下面会有例子），redo 日志中只需要记录一下在某个页面的某个偏移量处修改了几个字节的值、具体修改后的内容是啥什么就好了，比如操作 Max Row Id。
- MLOG_1BYTE: 表示在页面的某个偏移量处写入 1 字节的 redo 日志类型。
- MLOG_2BYTE: 表示在页面的某个偏移量处写入 2 字节的 redo 日志类型。
- MLOG_4BYTE: 表示在页面的某个偏移量处写入 4 字节的 redo 日志类型。
- MLOG_8BYTE: 表示在页面的某个偏移量处写入 8 字节的 redo 日志类型。

类型	表空间ID	页号	页面中的偏移量	redo日志的具体内容
----	-------	----	---------	-------------

- MLOG_WRITE_STRING: 表示在页面的某个偏移量处写入一个字节序列。

类型	表空间ID	页号	页面中的偏移量	数据占用的字节数	redo日志的具体内容
----	-------	----	---------	----------	-------------

redo复杂日志类型——例：MLOG_COMP_REC_INSERT



MLOG_REC_INSERT (type=9)

MLOG_COMP_REC_INSERT (type=38)

MLOG_COMP_PAGE_CREATE (type=58)

MLOG_COMP_REC_DELETE (type=42)

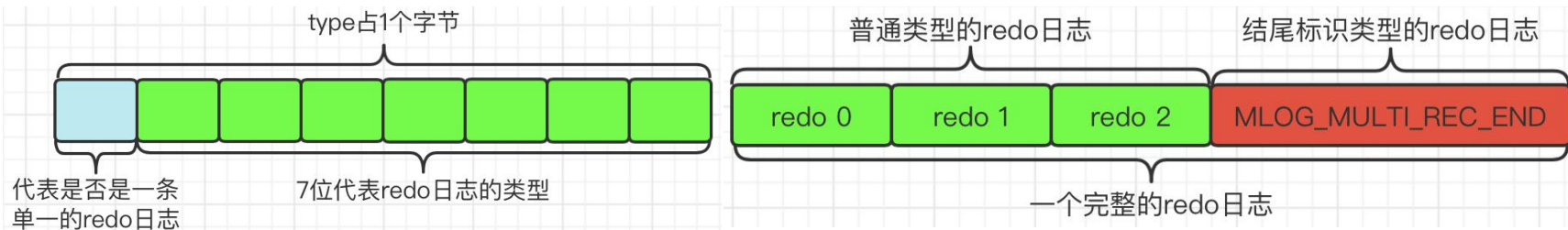
MLOG_COMP_LIST_START_DELETE (type=44)

MLOG_COMP_LIST_END_DELETE (type=43)

MLOG_ZIP_PAGE_COMPRESS (type=51)

redo 日志组

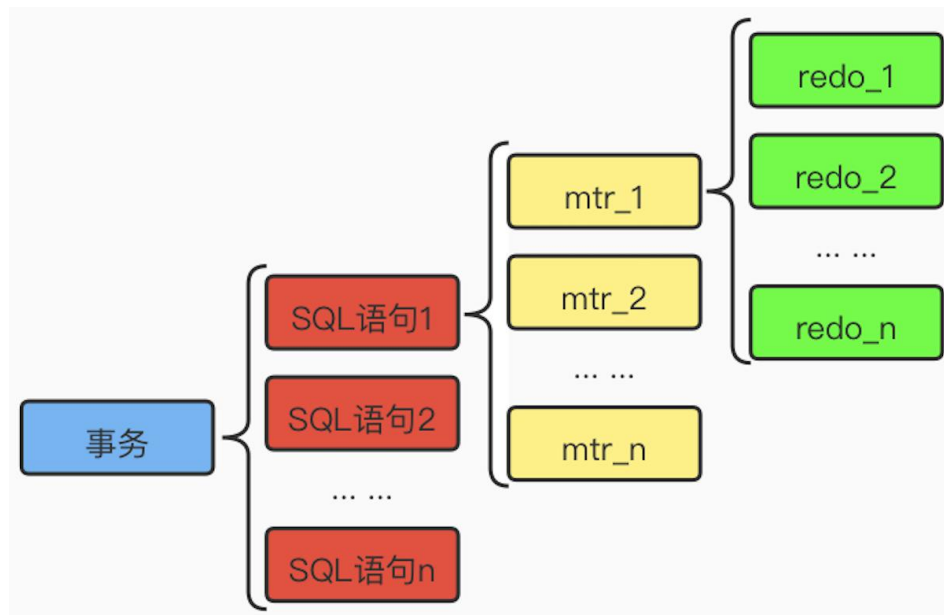
- 在执行语句的过程中产生的redo日志，被InnoDB划分成了若干个不可分割的组。比如：更新Max Row ID属性时产生的redo日志为一组，是不可分割的；向聚簇索引/二级索引对应B+树的页面中插入一条记录时产生的redo日志为一组，是不可分割的等等。
- InnoDB认为，比如向某个索引对应的B+树中插入一条记录的过程必须是原子的，不能说插入了一半之后就停止了。否则就会形成一棵不正确的B+树。所以他们规定在执行这些需要保证原子性的操作时，必须以组的形式来记录redo日志。在进行恢复时，针对某个组中的redo日志，要么把全部的日志都恢复，要么一条也不恢复。



MTR (Mini-Transaction)

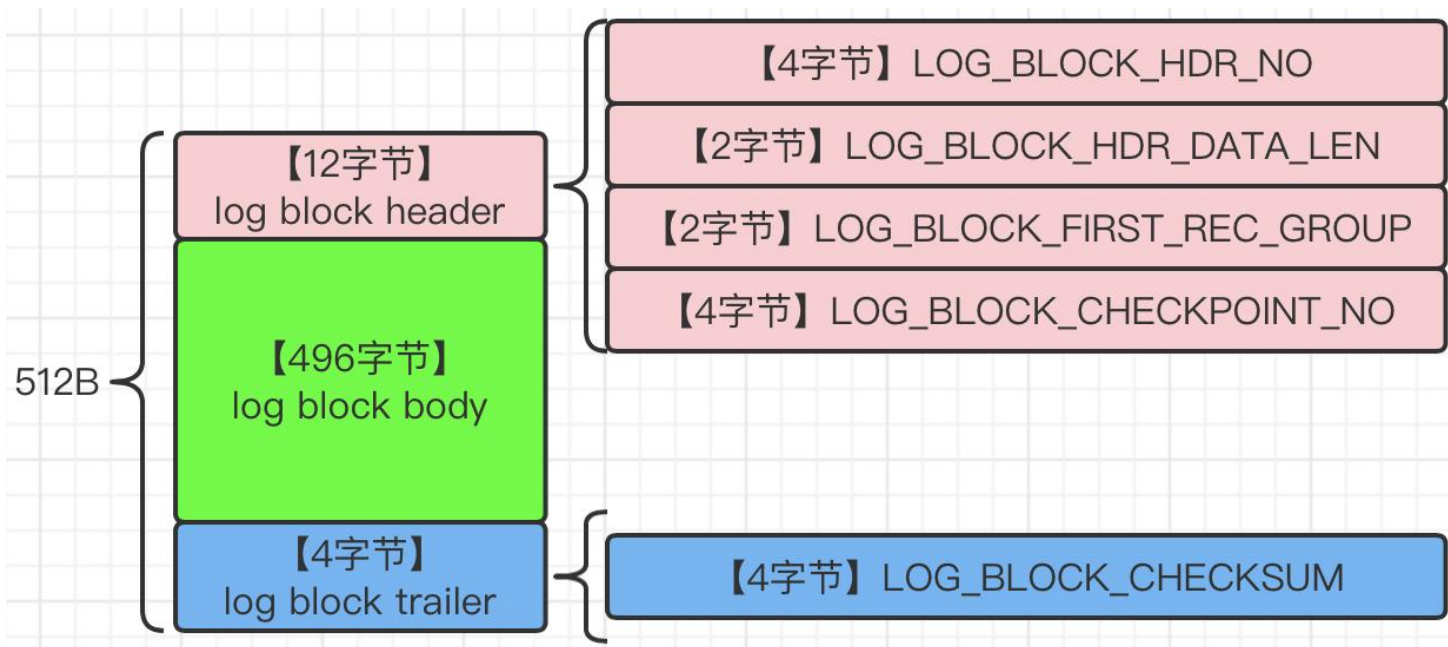
- 对底层页面进行一次原子访问的过程被称为一个Mini-Transaction (MTR)。
- 事务、语句、MTR、redo日志之间的关系，如下所示：

- ① 1个事务可以包含N条SQL语句
- ② 1条SQL语句可以包含N个MTR
- ③ 1条MTR可以包含N条redo日志



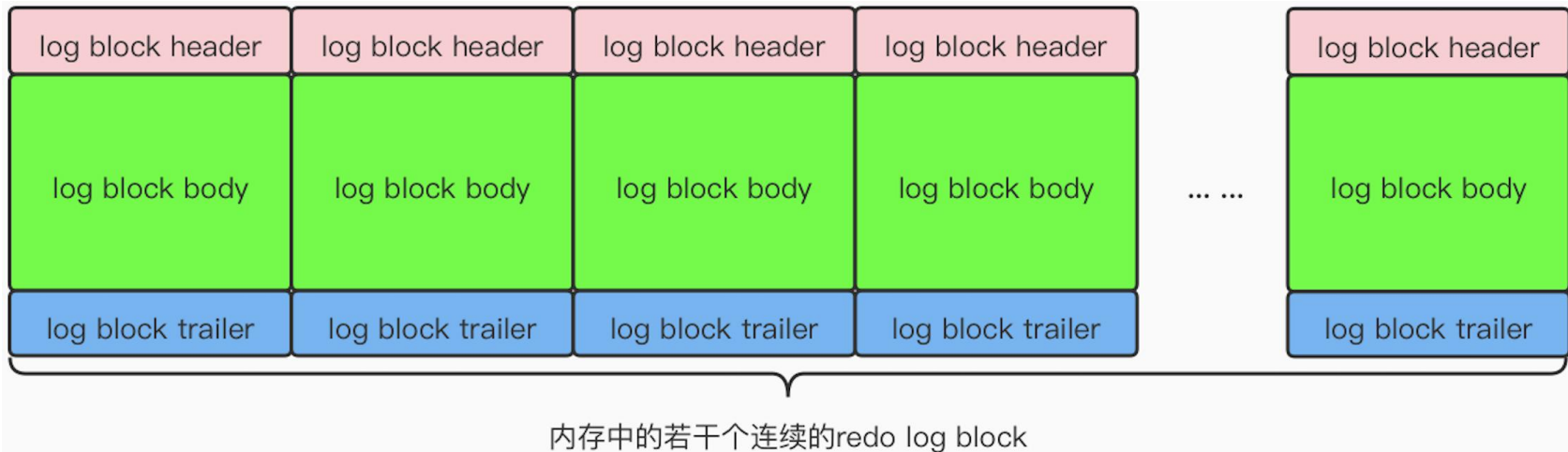
redo log block

- 为了更好地管理redo日志，InnoDB把通过MTR生成的redo日志都放在了大小为**512字节**的页中，把用来存储redo日志的页称为**block**。



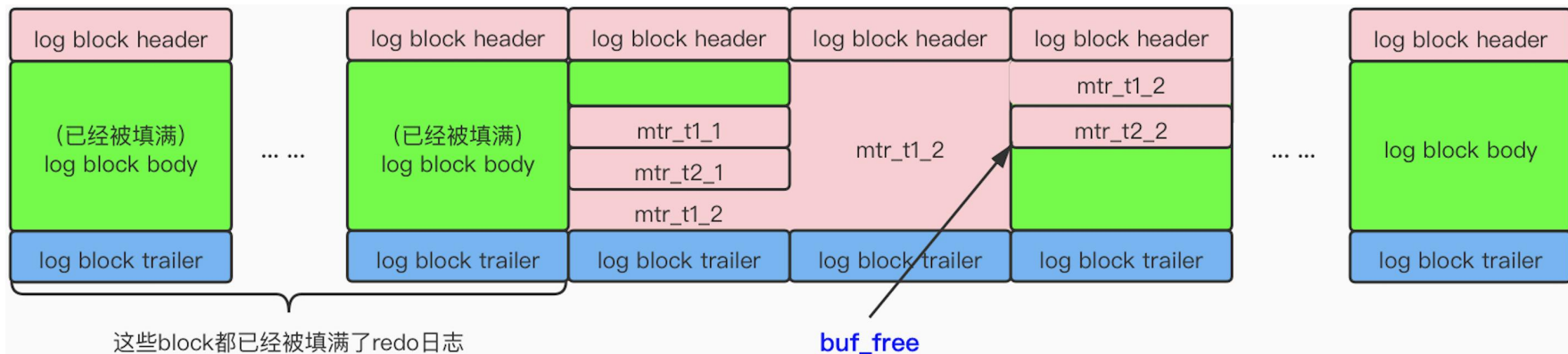
redo日志缓冲区——log buffer

- 与Buffer Pool类似，写入redo日志时也不能直接写到磁盘中，实际上在服务器启动时就向操作系统申请了一大片称为redo log buffer (redo日志缓冲区) 的连续内存空间，也可以将其简称为log buffer。这片内存空间被划分成若干个连续的redo log block。
- 其中，用innodb_log_buffer_size来指定log buffer的大小。该启动选项的默认值为16MB。



redo 日志写入 log buffer

- 向 log buffer 中写入 redo 日志的过程是**顺序写入**的。其中，**buf_free** 是一个全局变量，该变量指明后续写入的 redo 日志应该写到 log buffer 中的哪个位置。
- 一个 MTR 执行过程中可能产生若干条 redo 日志，这些 redo 日志是一个不可分割的组，所以**并不是每生成一条 redo 日志就将其插入到 log buffer 中**，而是将每个 MTR 运行过程中产生的日志先暂时存到一个地方，当该 MTR 结束的时候，再将过程中产生的一组 redo 日志全部复制到 log buffer 中。



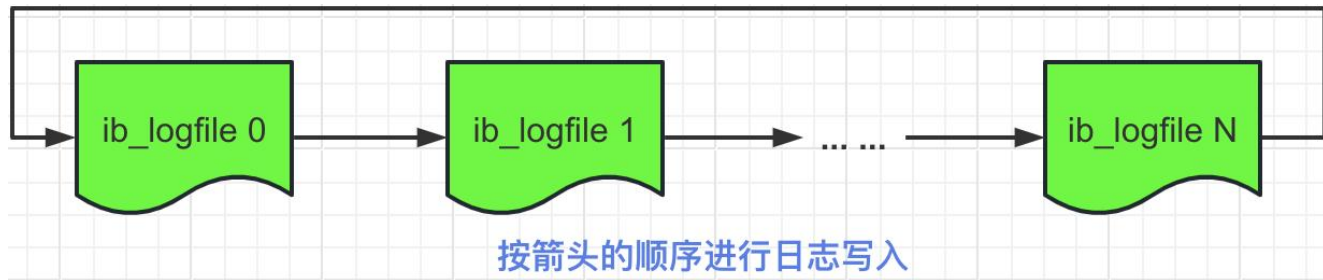
不同的事务是可能并发执行的，所以 T1、T2 的 MTR 可能是交替执行的。

redo日志刷盘和日志文件组

➤ MTR运行过程中产生的一组redo日志在MTR结束时会被复制到log buffer中。可是这些日志总在内存里也不是办法，在一些情况下它们会被刷新到磁盘中。

- ① log buffer空间不足50%的时候
- ② 事务提交的时候
- ③ 后台有线程，大约以每秒1次的频率将log buffer中的redo日志刷新到磁盘。
- ④ 正常关闭服务器时
- ⑤ 做checkpoint时

➤ 在MySQL的数据目录中，默认有名称为：ib_logfile0和ib_logfile1的两个文件，log buffer中的日志在默认情况下就是刷新到这两个磁盘文件中，也可以通过下一页中的配置参数对其进行调节和修改：



查看redo日志相关配置信息

- **datadir:** 查看数据目录所在位置。

```
mysql> show variables like 'datadir';
+-----+-----+
| Variable_name | Value                |
+-----+-----+
| datadir       | /usr/local/mysql/data/ |
+-----+-----+
1 row in set (0.01 sec)
```

```
muse@muse:/usr/local/mysql/data> ll /usr/local/mysql/data/ | grep ib_logfile
-rw-r----- 1 _mysql _mysql 50331648 9 21 19:42 ib_logfile0
-rw-r----- 1 _mysql _mysql 50331648 9 21 19:42 ib_logfile1
```

- **innodb_log_group_home_dir:** 指定了redo日志文件所在的目录，默认值为当前的数据目录。

```
mysql> show variables like 'innodb_log_group_home_dir';
+-----+-----+
| Variable_name          | Value |
+-----+-----+
| innodb_log_group_home_dir | ./    |
+-----+-----+
1 row in set (0.00 sec)
```

查看redo日志相关配置信息

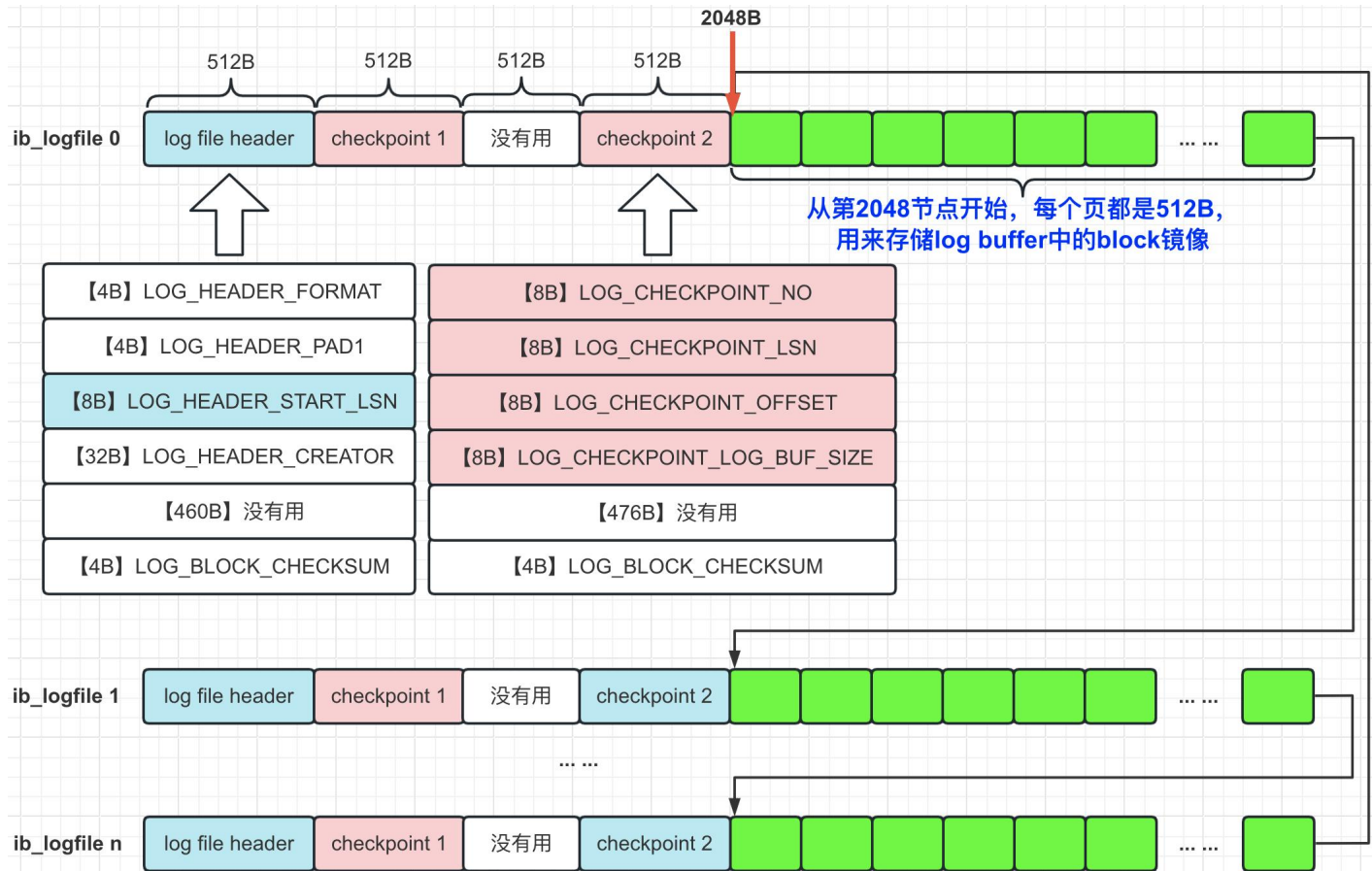
- **innodb_log_file_size**: 指定了每个redo日志文件的大小，默认值为**48MB**。

```
mysql> show variables like 'innodb_log_file_size';
+-----+-----+
| Variable_name | Value |
+-----+-----+
| innodb_log_file_size | 50331648 |
+-----+-----+
1 row in set (0.00 sec)
```

- **innodb_log_files_in_group**: 指定了redo日志文件的个数，默认值为2，最大值为**100**个。

```
mysql> show variables like 'innodb_log_files_in_group';
+-----+-----+
| Variable_name | Value |
+-----+-----+
| innodb_log_files_in_group | 2 |
+-----+-----+
1 row in set (0.01 sec)
```

redo日志文件格式



lsn (log sequence number)

➤ lsn全称为log sequence number，是一个全局变量，用来记录当前总共已经写入的redo日志量。

➤ lsn初始值为8704，也就是说，一条redo日志什么也没写入的时候，lsn的值就是8704。



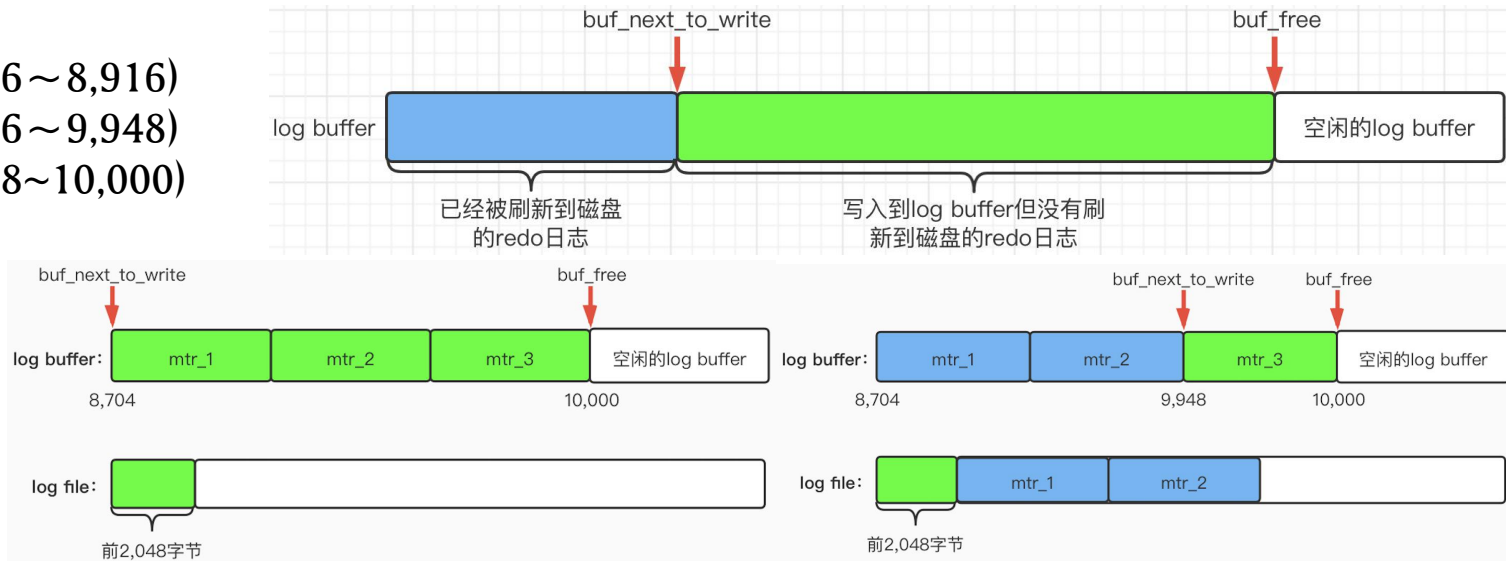
redo日志刷新到磁盘

mtr_1(8,716~8,916)

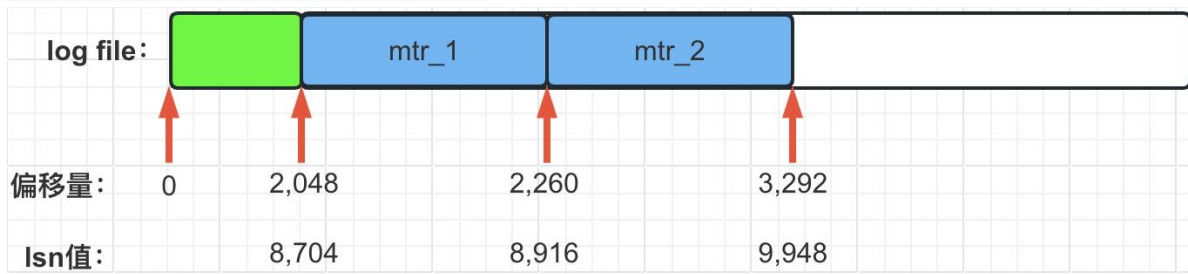
mtr_2(8,916~9,948)

mtr_3(9,948~10,000)

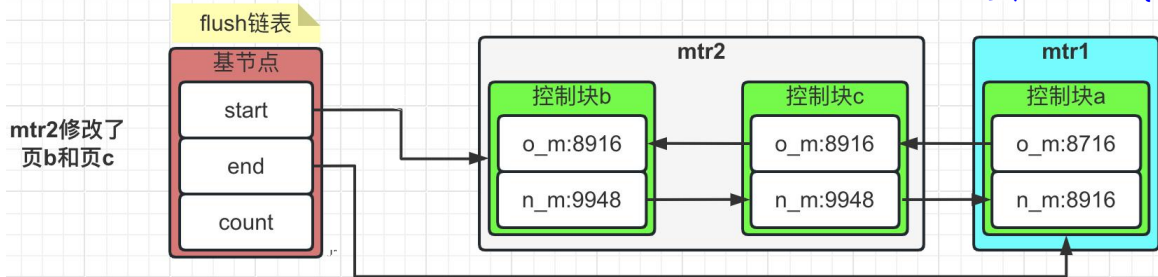
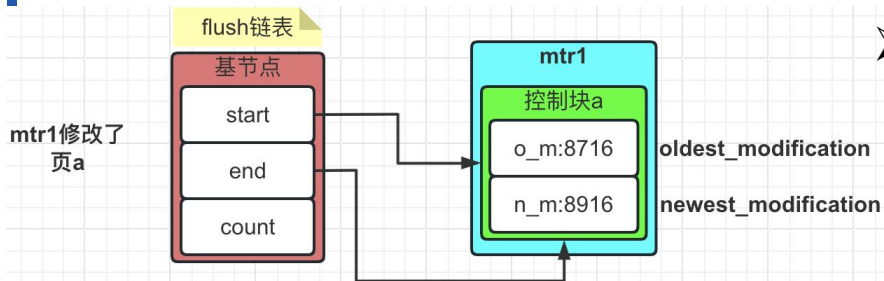
文件写入 演示



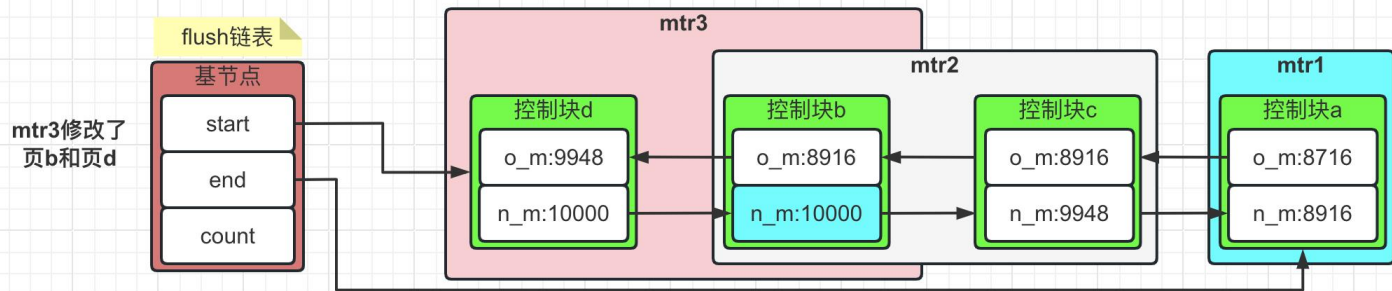
lsn值和redo日志文件组 偏移量的对应关系



flush 链表

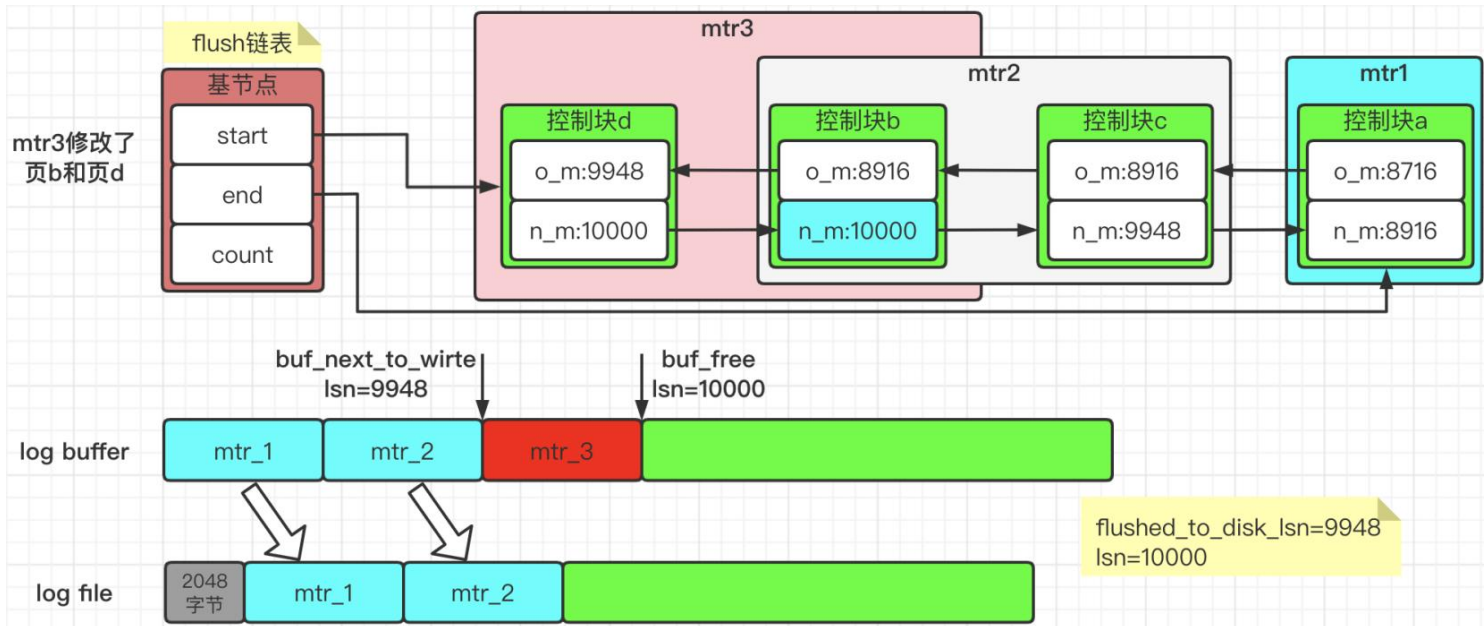


页b第二次被修改了，所以只改变 newest_modification 的值，oldest_modification 的值不变化。



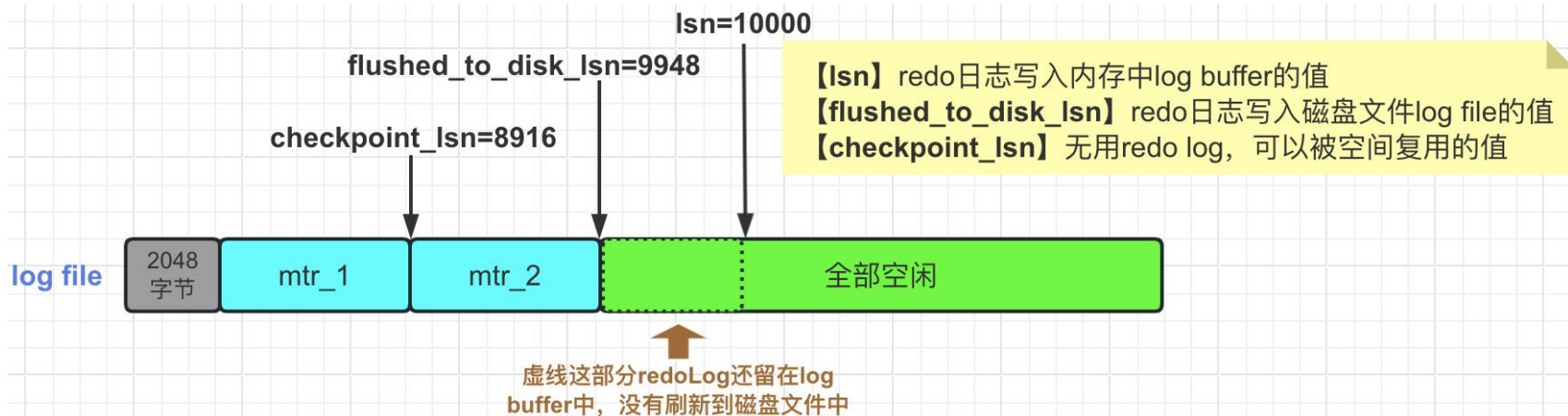
刷入磁盘

- mtr1和mtr2生成的redo日志虽然已经写到磁盘上的log file中了，但是它们修改的脏页仍然留在Buffer Pool中，所以它们的redo日志不可以被覆盖。随着系统运行，如果页a从Buffer Pool中刷到了磁盘上，那么页a对应的控制块就会从flush链表中移除掉。而且，它的redo日志占用的空间就可以被覆盖掉了。



checkpoint

- InnoDB通过全局变量`checkpoint_lsn`，来表示当前系统中可以被覆盖的redo日志总量是多少。这个变量的初始值也是8704（因为lsn的初始值就是8704）。比如，现在页a被刷新到了磁盘上，mtr1生成的redo日志就可以被覆盖了，所以进行一个增加`checkpoint_lsn`的操作。我们把这个过程称为执行一次`checkpoint`。
- redo日志文件组中各个lsn值的关系，如下图所示：



innodb_flush_log_at_trx_commit

- 为了保证事务的持久性，用户线程在事务提交时，需要将该事务执行过程中产生的所有redo日志都刷新到磁盘中。
- 这个规则我们可以通过系统变量`innodb_flush_log_at_trx_commit`来进行配置修改，该变量有如下3个可选值：
 - 0：在事务提交时，不立即向磁盘同步redo日志，这个任务交给后台线程来处理；
 - 1：在事务提交时，需要将redo日志同步到磁盘。（默认值）
 - 2：在事务提交时，需要将redo日志写到操作系统的缓冲区中，但并不需要保证将日志真正刷新到磁盘。如果操作系统挂掉了，则数据丢失。

```
mysql> show variables like 'innodb_flush_log_at_trx_commit';
+-----+-----+
| Variable_name | Value |
+-----+-----+
| innodb_flush_log_at_trx_commit | 1     |
+-----+-----+
```

undo 日志

什么是undo log

- 事务是需要保证原子性的，也就是说，事务中的操作要么全部完成，要么什么也不做。但有如下情况，会造成事务执行不完：
 - ① 事务执行过程中可能遇到各种错误，比如：服务器宕机，操作系统异常，突然断电……
 - ② 程序员在事务执行过程中手动输入rollback语句结束当前事务的执行。
- 遇到上面的情况，为了保证事务的原子性，我们需要把数据还原回原来的样子，这个过程就叫做回滚 (rollback)
- 数据库为了回滚而记录的日志，我们就称之为撤销日志 (undo log)
- 注意一点，由于SELECT操作并不会修改任何记录，所以并不需要记录相应的undo日志

如何开启事务

开启的事务类型	解释
只读事务	通过 <code>START TRANSACTION READ ONLY</code> 语句开启一个只读事务。在只读事务中，不可以对普通表进行增删改操作；但可以对临时表进行增删改操作。
读写事务	通过 <code>START TRANSACTION READ WRITE</code> 语句开启一个读写事务。 使用 <code>BEGIN</code> 、 <code>START TRANSACTION</code> 语句开启的事务，默认也算是读写事务。 在读写事务中可以对表执行增删改查操作。

何时分配事务id

- 只有在事务对表中的记录进行改动时才会为这个事务分配一个唯一的事务id, 否则事务id值默认为0。
- 只读事务何时分配事务id?
只有在它第一次对某个用户创建的临时表 (CREATE TEMPORARY TABLE) 执行增删改操作时, 才会为这个事务分配一个事务id, 否则是不分配的。
- 读写事务何时分配事务id?
只有在它第一次对某个表 (包括用户创建的临时表) 执行增删改操作时, 才会为这个事务分配一个事务id, 否则是不分配的。

事务id是怎么生成的

➤ 事务id本质上就是一个数字，事务id生成策略如下：

- ① 内存中维护一个全局变量，每当需要为某个事务分配事务id时，就会把该变量值当作事务id分配给该事务，并且自增1。
- ② 每当这个变量的值为256的倍数时，就会将值刷新到系统表空间中页号为5的页面中一个名为Max Trx ID的属性中（占用8个字节）。
- ③ 当系统下一次启动时，会将Max Trx ID的值加载到内存中，并加上256之后赋值给前面提到的全局变量。

➤ 为什么要加256？

答：因为上次关机时，该全局变量的值可能大于磁盘页面中的Max Trx ID属性值。

事务id在记录中存储的位置

- `trx_id`表示对该条记录进行改动的语句所对应的**事务id**。



- 我们下面来看看，对表中数据进行不同操作都会产生什么样的undo日志？但是在此之前，我们先创建一张表作为下面实验用的数据表：

tb_user table_id=1065		
id<int> (主键)	name<varchar(100)> (二级索引)	city<varchar(100)>

- 不同版本下，查询**`table_id`**的方式：

【mysql 5.x】`select * from information_schema.innodb_sys_tables;`

【mysql 8.x】`select * from information_schema.innodb_tables;`

INSERT对应的undo日志结构

➤ TRX_UNDO_INSERT_REC类型的undo日志结构:

end of record	undo type	undo no	table id	主键各列信息 <len, value>列表	start of record
---------------	-----------	---------	----------	--------------------------	-----------------

名称	解释
end of record	undoLog结束，下一条开始时在页面中的地址。
undo type	undoLog的类型，也就是TRX_UNDO_INSERT_REC。
undo no	undoLog对应的编号，在一个事务中是从0开始递增的，只要事务没提交，每生成一条undo日志，那么该条日志的undo no就加1。
table id	undoLog对应的记录所在表的table_id。
主键各列信息 <len, value>列表	主键的每个列占用的存储空间大小和真实值，如果记录中的主键包含多个列，那么每个列占用的存储空间大小和对应的真实值都需要记录下来。
start of record	上一条undoLog结束，本条开始时在页面中的地址

INSERT对应的undo日志操作演示

➤ 我们先插入两条记录:

```
# 显示开启一个事务，假设该事务的事务id为100  
BEGIN;
```

```
# 插入两条记录
```

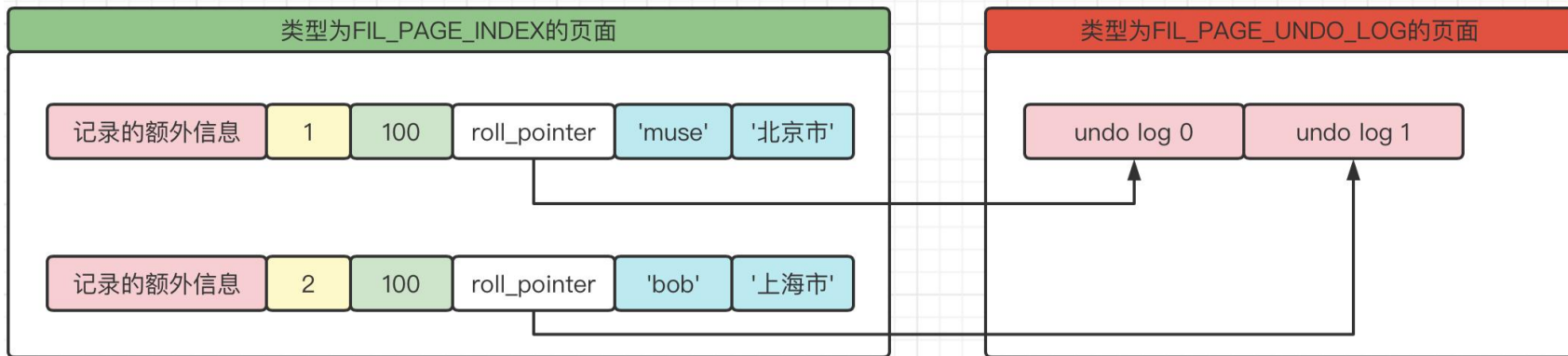
```
INSERT INTO tb_user(id, name, city) VALUES(1, 'muse','北京市'), (2, 'bob','上海市');
```

➤ undoLog日志内容为:

结束地址	TRX_UNDO_INSERT_REC	0	1065	<4, 1>	开始地址
结束地址	TRX_UNDO_INSERT_REC	1	1065	<4, 2>	开始地址

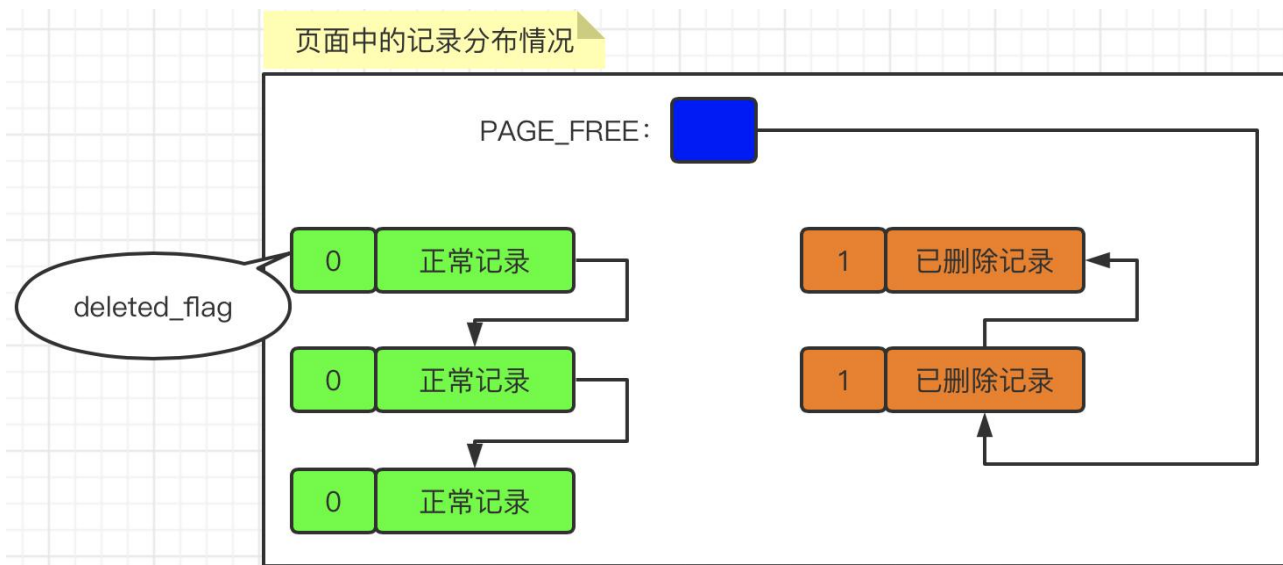
INSERT操作对应的undo日志

- roll_pointer本质上就是一个指向记录对应的undo日志的指针。不同的内容放到了不同的页面中，其中：
 - ① 聚簇索引记录存放类型到类型为FIL_PAGE_INDEX的页面；
 - ② undo日志存放类型到类型为FIL_PAGE_UNDO_LOG的页面；
- 聚簇索引记录和undo日志的存放位置，如下图所示：



DELETE操作之垃圾链表

- 被删除的记录其实也会根据记录头信息中的`next_record`属性组成一个链表，只不过这个链表中的记录所占用的存储空间可以被重新利用，所以也称这个链表为**垃圾链表**。Page Header部分中有一个名为**PAGE_FREE**的属性，它指向由被删除记录组成的垃圾链表中的**头节点**。每删除一条记录，则该记录都会插入到垃圾链表的头节点处。
- 操作演示，有**3条正常记录**和**2条被删除记录**，他们在页中的记录分布情况如下所示：

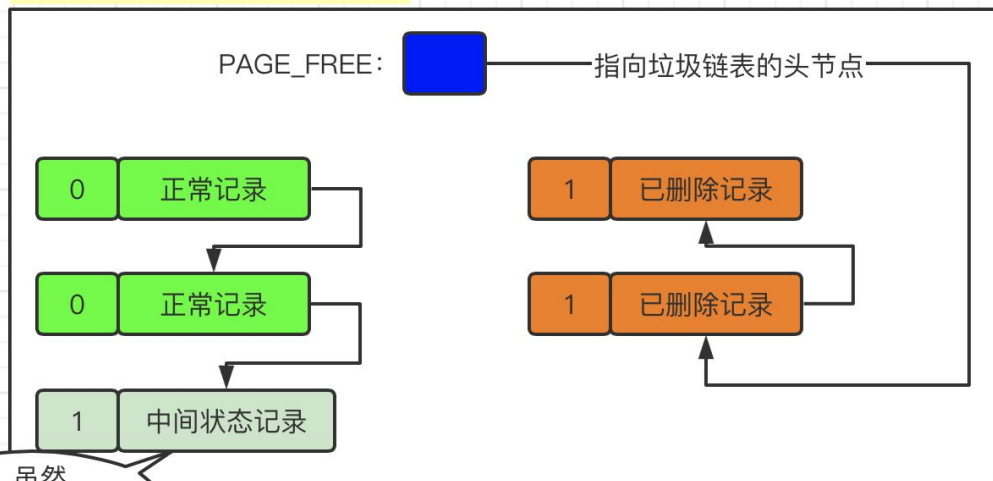


删除记录的两个步骤

➤ 第一步：delete mark阶段

仅仅将记录的deleted_flag标识位设置为1，但是这条记录并没有加入到垃圾链表中。也就是说，这条记录即不是正常记录，也不是已删除记录。在删除语句所在的事务提交之前，被删除的记录一直都处于这种中间状态（其实主要是为了实现MVCC的功能才这样处理的）。

delete mark执行过程示意图

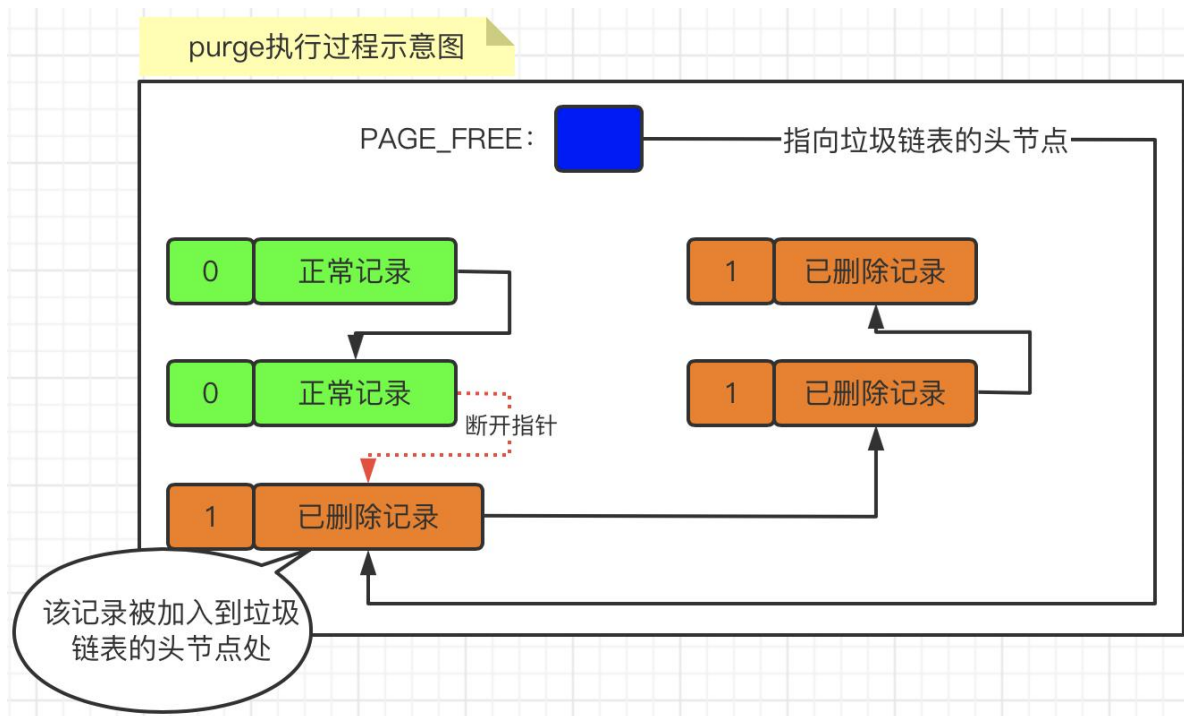


虽然
deleted_flag=1, 但
是却没有被移动到垃
圾链表

删除记录的两个步骤

➤ 第二步: **purge**阶段

当该删除语句所在的事务提交后, 会有专门的线程来把该记录从正常记录链表中移除, 并**加入**到垃圾链表中作为**头节点**。



DELETE操作对应的undoLog结构

➤ TRX_UNDO_DEL_MARK_REC类型的undo日志结构:

TRX_UNDO_DEL_MARK_REC类型的undo日志结构

end of record	undo type	undo no	table id	info bits	trx_id	roll_pointer	主键各列信息 <len, value>列表	len of index_col_info	索引列各列信息 <pos, len, value>列表	start of record
---------------	-----------	---------	----------	-----------	--------	--------------	--------------------------	-----------------------	--------------------------------	-----------------

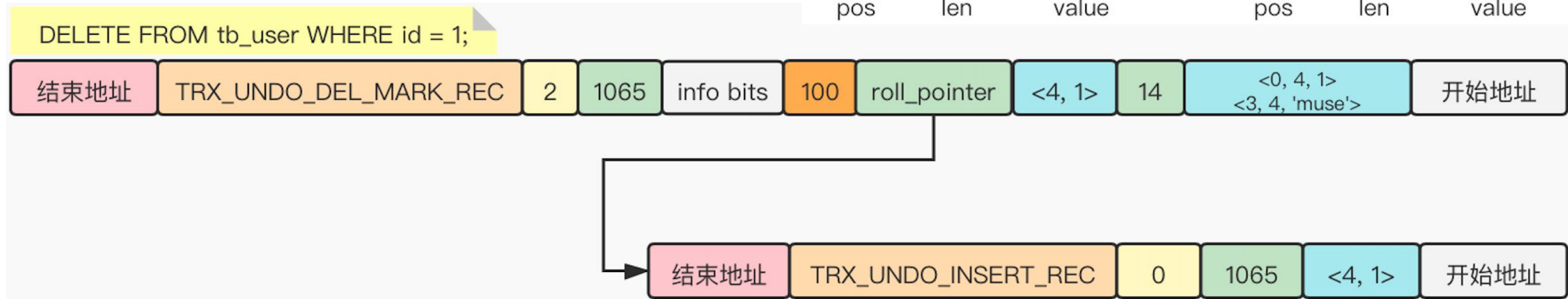
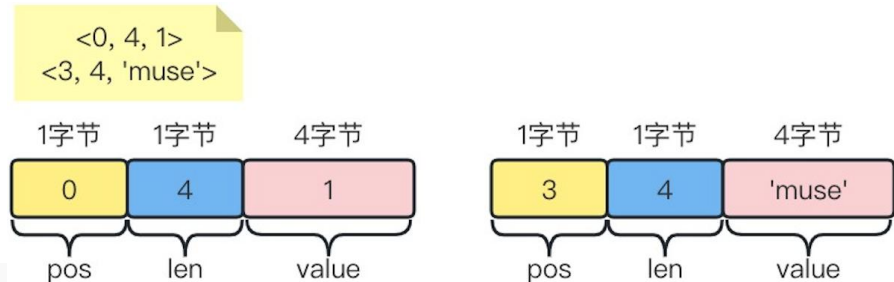
名称	解释
info bits	记录头信息的前4个比特的值。
trx_id	旧记录的trx_id值。
roll_pointer	旧记录的roll_pointer值。
len of index_col_info	也就是下边的【索引列各列信息】部分和本部分占用的存储空间总和。
索引列各列信息 <pos, len, value>列表	凡是被索引的列的各列信息。

DELETE操作对应的undo日志

➤ 删除id等于1的这条记录:

删除一条记录

```
DELETE FROM tb_user WHERE id = 1;
```



UPDATE操作对应的undoLog结构

➤ TRX_UNDO_UPD_EXIST_REC类型的undo日志结构:

TRX_UNDO_UPD_EXIST_REC类型的undo日志结构

end of record	undo type	undo no	table id	info bits	trx_id	roll_pointer	主键各列信息 <len, value>列表	n_updated	被更新的列更新前信息 <pos, old_len, old_value>列表	len of index_col_info	索引列各列信息 <pos, len, value>列表	start of record
---------------	-----------	---------	----------	-----------	--------	--------------	--------------------------	-----------	---	-----------------------	--------------------------------	-----------------

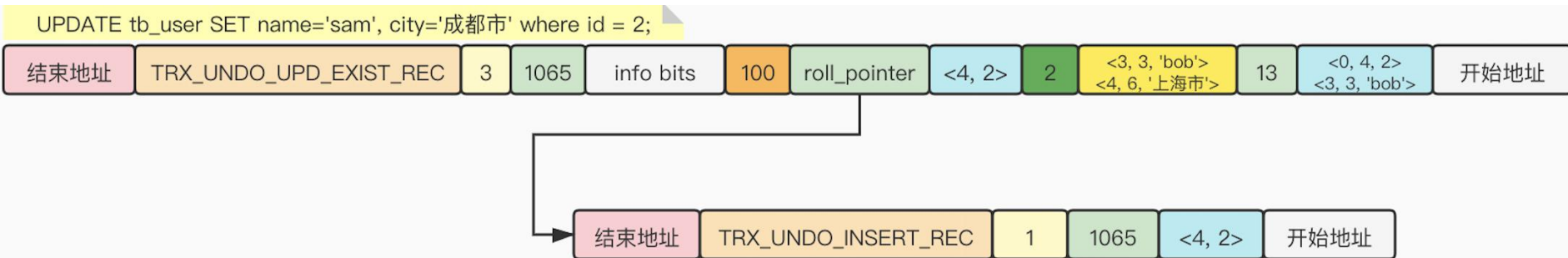
名称	解释
n_updated	表示本条UPDATE语句执行后将有几个列被更新。
被更新的列更新前信息 <pos, old_len, old_value> 列表	被更新列在记录中的位置、更新前该列占用的存储空间大小、更新前该列的真实值。

UPDATE操作对应的undo日志

➤ 更新id等于2的这条记录:

更新一条记录

```
UPDATE tb_user SET name='sam', city='成都市' where id = 2;
```



其中， $\langle 0, 4, 2 \rangle \langle 3, 3, 'bob' \rangle$ 占用空间等于： $(1+1+4)+(1+1+3)=11$ ，最后，再加上 len of index_col_info 属性本身占2个字节，所以总共13字节。 即：len of index_col_info=13。

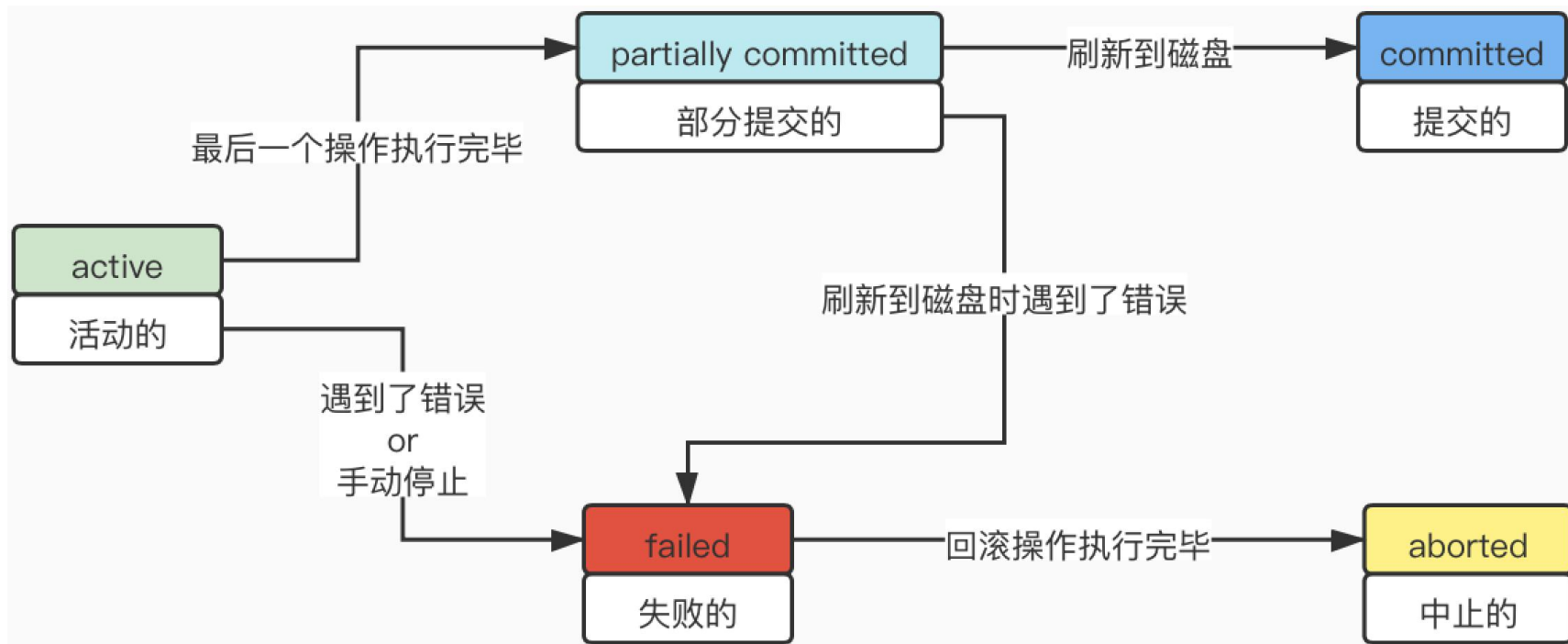


事务

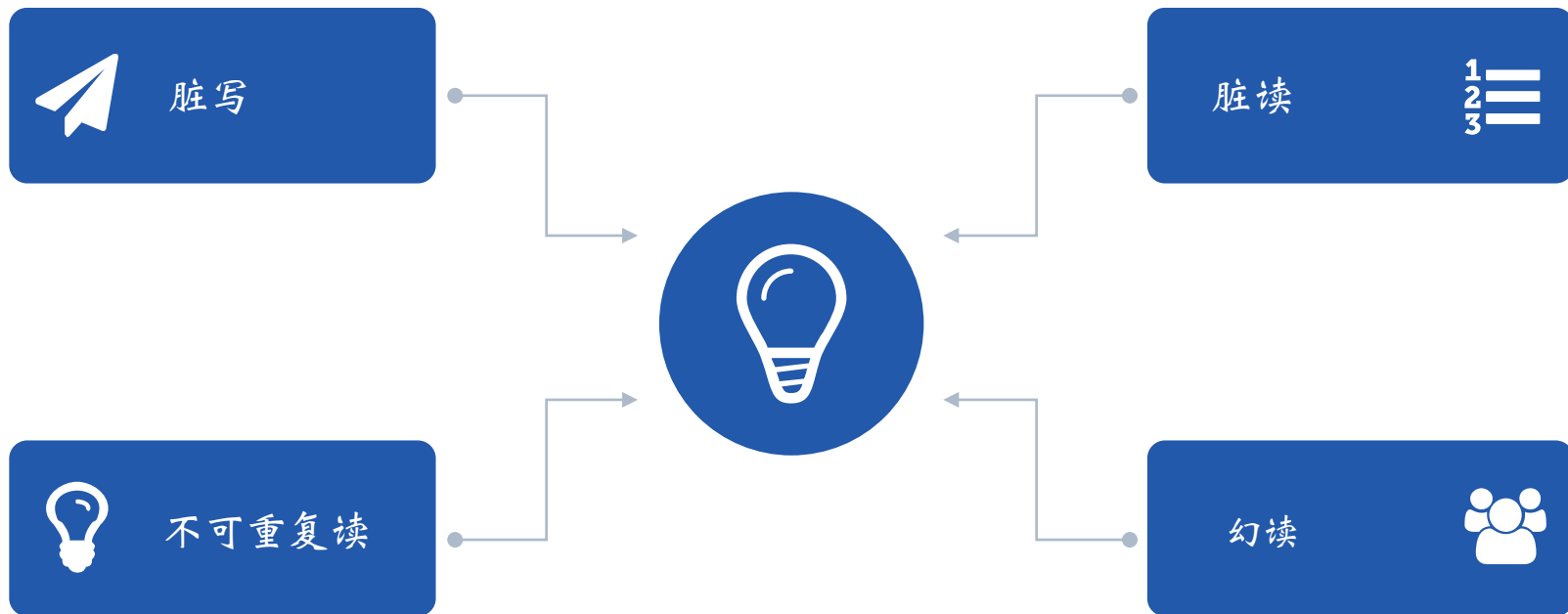
什么是ACID?

- **Atomicity** 原子性
某个操作，要么全都执行完毕，要么全都回滚。
- **Consistency** 一致性
数据库中的数据全都符合现实世界中的约束，则这些数据就符合一致性。
比如性别约束男or女；人民币面值不能为负数；出生地址不能为null；参与转账的账户总余额不变；等等。
- **Isolation** 隔离性
多个事务访问相同数据时，对该数据不同状态的转换对应的数据库操作的执行顺序有一定的规律，彼此不干涉。
- **Durability** 持久性
现实中的状态转换映射到数据库中，意味着对数据所做的修改都应该在磁盘中保存。

事务的状态有哪些？

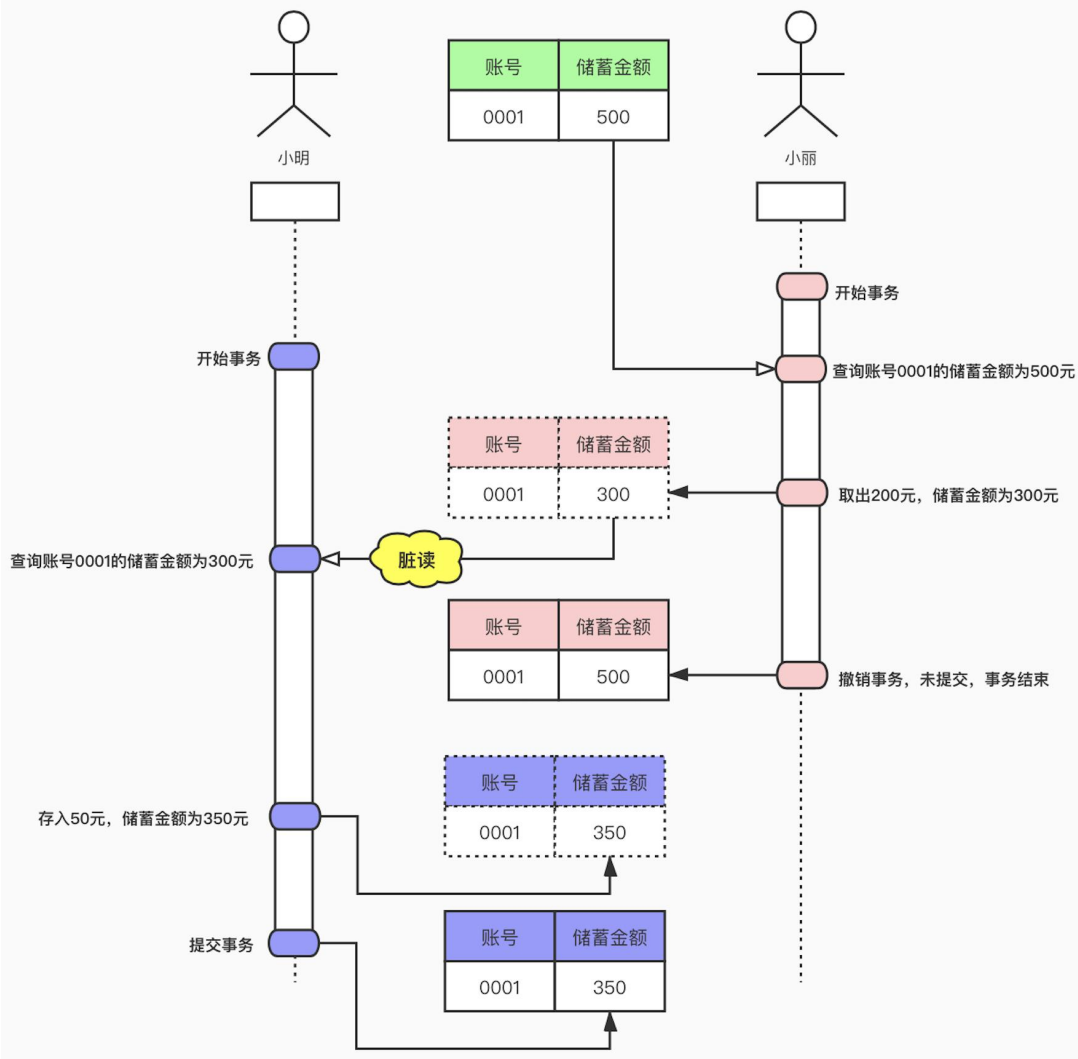


事务并发执行时数据一致性问题有哪些？



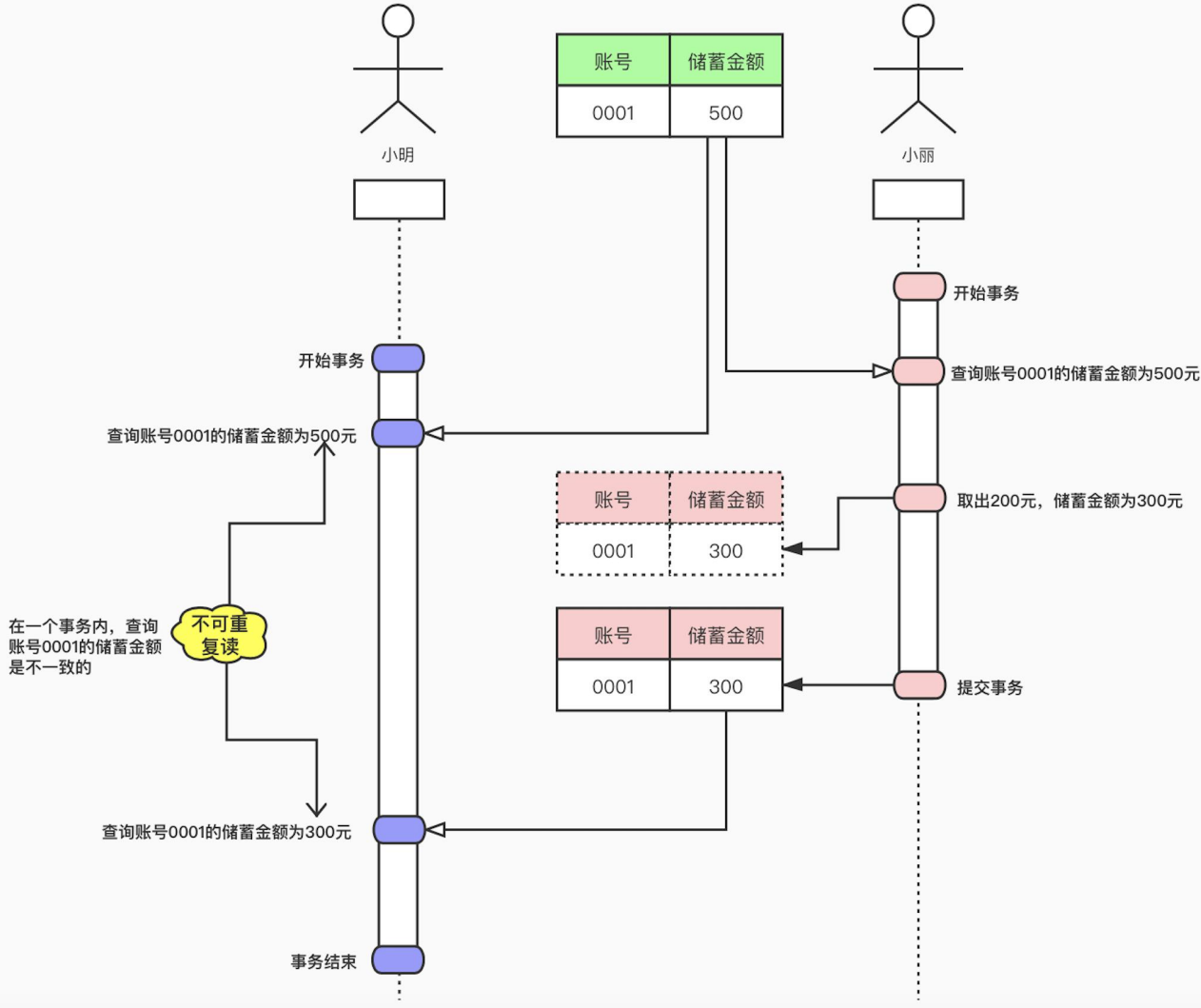
脏读

- 如果一个事务（小明）读取到了另一个未提交事务（小丽）修改过的数据，就意味着发生了脏读现象。



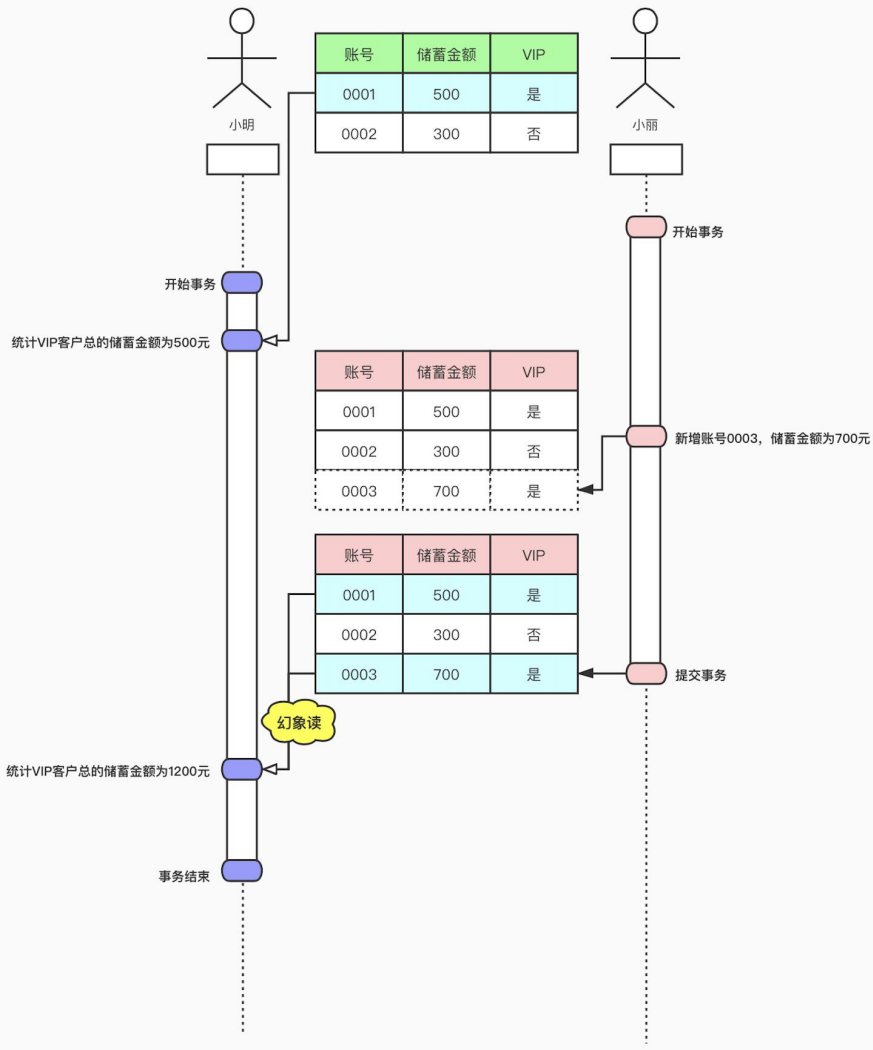
不可重复读

- 如果一个事务（小丽）修改了另一个未提交事务（小明）读取的数据，就意味着发生了不可重复读现象，或者叫模糊读FuzzyRead



幻象读

- 如果一个事务（小明）先根据某些搜索条件（`select ... where vip='是'`）查询了一些记录，但是在该事务并未提交时，另一个事务（小丽）写入了一些符合上面搜索条件的记录（这里的写入可以是 insert、delete、update 操作）。例如：
`insert into ... values('0003',700,'是')`，就意味着发生了幻读现象。



SQL 事务隔离级别

隔离级别	脏读	不可重复读	幻象读
READ UNCOMMITTED	Y	Y	Y
READ COMMITED	N	Y	Y
REPEATABLE READ	N	N	Y
SERIALIZABLE	N	N	N

由于无论哪种隔离级别，都不允许脏写的情况发生，所以没有列入到表格中。



MVCC

MVCC&ReadView

➤ MVCC (Multi-Version Concurrency Control)

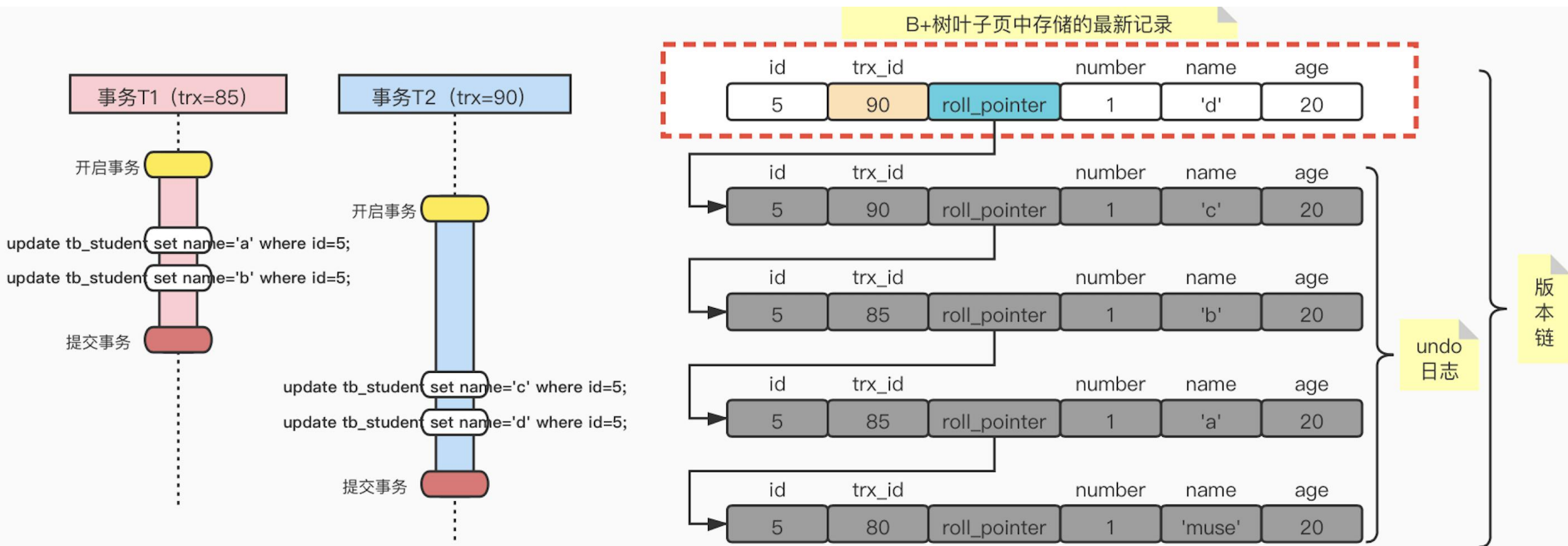
多版本并发控制，利用记录的版本链和ReadView，来控制并发事务访问相同记录时的行为。

➤ ReadView

一致性视图，用来判断版本链中的哪个版本是当前事务可见的。

版本链

- 在每次更新该记录后，都会将旧值放到一条undo日志中。随着更新次数的增多，所有的版本都会被`roll_pointer`属性连接成一条链表，这个链表就称之为**版本链**。



ReadView 包含的内容

- ReadView也叫一致性视图，用来判断版本链中的哪个版本是当前事务可见的。ReadView包含4个比较重要的内容：

名称	解释
<code>m_ids</code>	在生成ReadView时，当前系统中活跃的读写事务的事务id列表。
<code>min_trx_id</code>	在生成ReadView时，当前系统中活跃的读写事务中最小的事务id，也就是m_ids中的最小值。
<code>max_trx_id</code>	在生成ReadView时，系统应该分配给下一个事务的事务id值。
<code>creator_trx_id</code>	生成该ReadView的事务的事务id。

- 只有在对表中的记录进行改动时（即：`insert`、`delete`、`update`）才会为事务分配唯一的事务id，否则一个事务的事务id值都默认为0。

如何通过ReadView来判断记录的某个版本是否可见？

- 如果 $RC.trx_id == RV.creator_trx_id$ ，则表明当前事务在访问它自己修改过的记录，所以该版本可以被当前事务访问。
- 如果 $RC.trx_id < RV.min_trx_id$ ，则表明生成该版本的事务在当前事务生成ReadView之前已经提交了，所以该版本可以被当前事务访问。
- 如果 $RC.trx_id \geq RV.max_trx_id$ ，则表明生成该版本的事务在当前事务生成ReadView之后才开启，所以该版本不可以被当前事务访问。
- 如果 $RC.trx_id \in RV.m_ids$ ，说明创建ReadView时生成该版本的事务还是活跃的，该版本不可以被访问。
- 如果 $RC.trx_id \notin RV.m_ids$ ，说明创建ReadView时生成该版本的事务已经被提交，该版本可以被访问（ m_ids 为空集合）。
- 如果某个版本的数据对当前事务不可见，那就顺着版本链找到下一个版本的数据，并继续执行上面的步骤来判断记录的可见性，以此类推，直到版本链中的最后一个版本。

ReadView生成的时机

tb_student		
id	name	age
5	'muse'	20

事务T1 (trx=10)

开启事务

update tb_student set name='a' where id=5;

update tb_student set name='b' where id=5;

事务T2 (trx=20)

开启事务

update tb_user set deleted=1 where id=1;

- 1> T1和T2都没有提交事务。
- 2> T2更新了其他表的数据。

【演示】

- 在READ COMMITTED隔离级别，数据的可见性
- 在REPEATABLE READ隔离级别，数据的可见性

B+树叶子页中存储的最新记录

id	trx_id	roll_pointer	name	age
5	10	roll_pointer	'b'	20

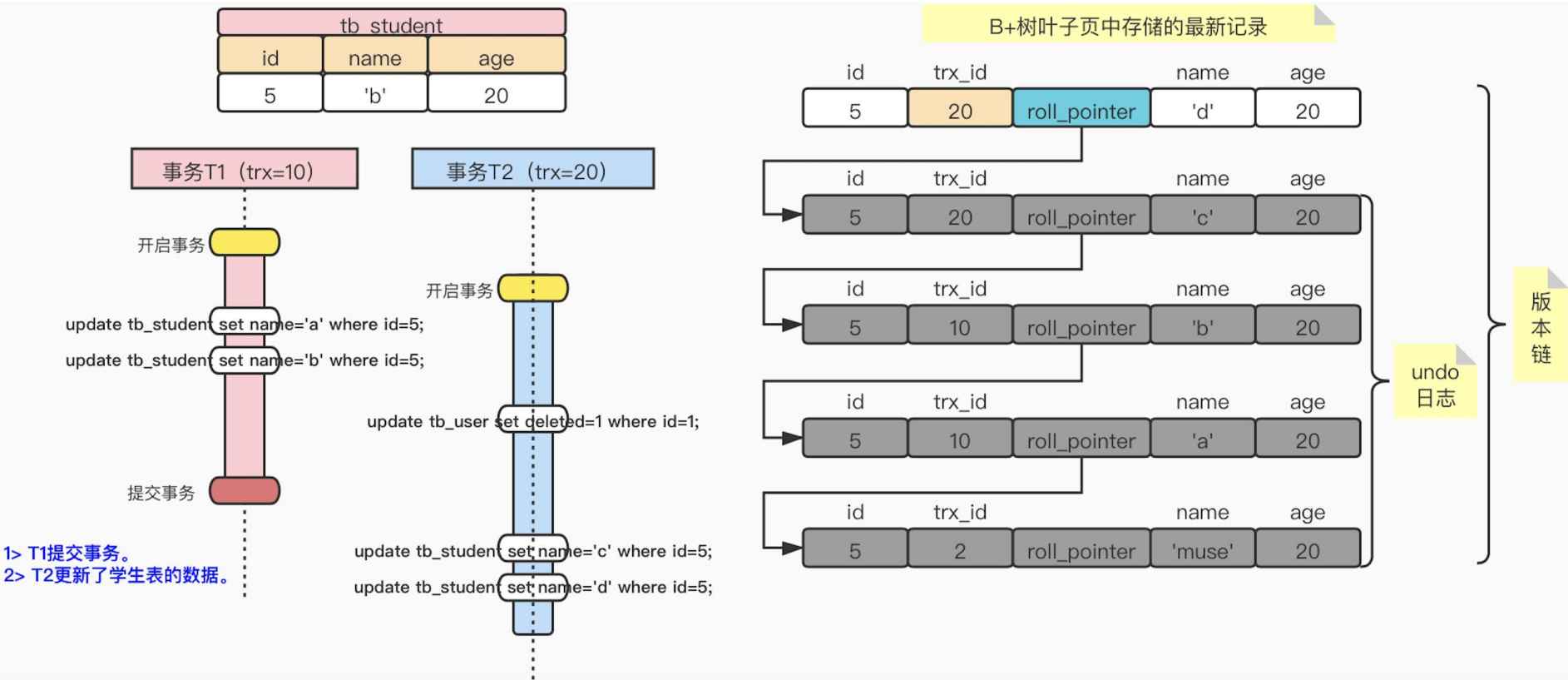
id	trx_id	name	age
5	10	'a'	20

id	trx_id	name	age
5	2	'muse'	20

undo
日志

版本链

ReadView生成的时机



ReadView 生成的时机

- READ COMMITTED和REPEATABLE READ隔离级别之间一个非常大的区别就是——它们生成ReadView的时机不同！！
- READ COMMITTED——在一个事务中，每次读取数据前都生成一个ReadView。
- REPEATABLE READ——在一个事务中，只在第一次读取数据时生成一个ReadView。

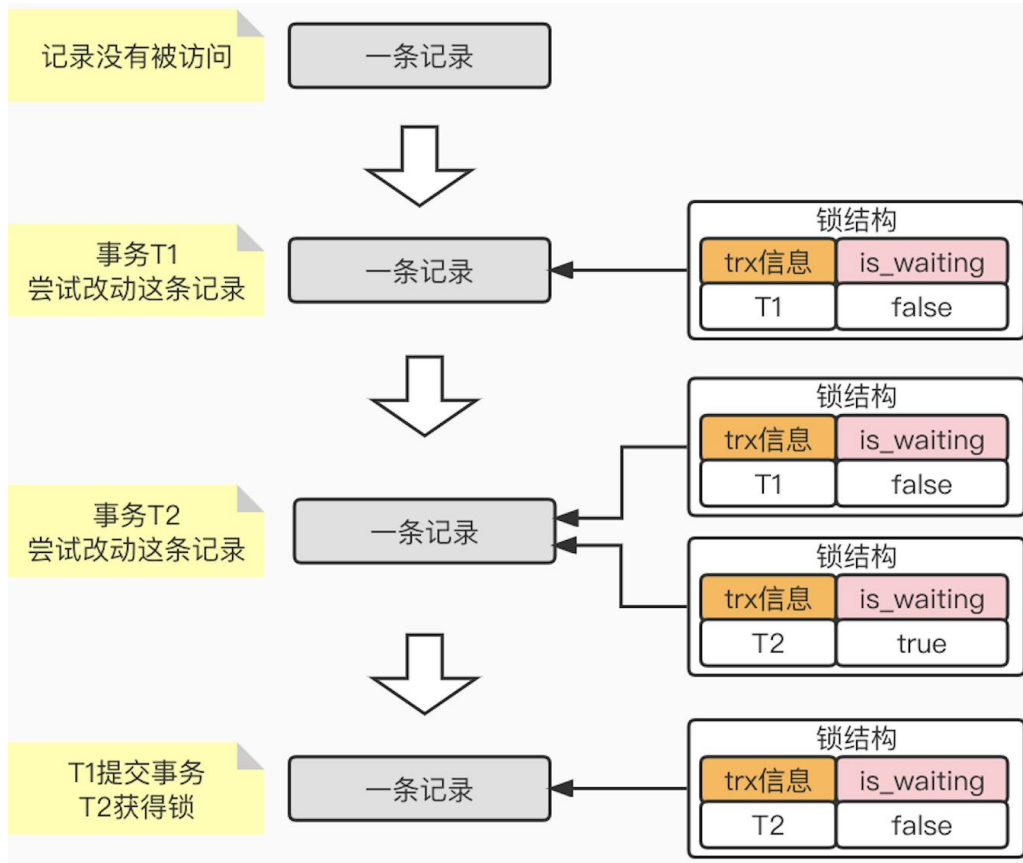
隔离级别及事务重要参数查询和设置

- `show variables like 'transaction_isolation';`
- `set session transaction isolation level read committed;`
- `set session transaction isolation level repeatable read;`
- `show variables like 'autocommit';`
- `set autocommit=0;`

锁

并发事务问题——写-写

- 由于任何一种隔离级别都不允许**脏写**（写-写）的现象发生，所以，当多个未提交事务相继对一条记录进行改动的时候，就需要让它们**排队执行**。
- 这个排队的过程其实是通过为该记录加锁来实现的。这个锁本质上是一个**内存中的结构**。



并发事务问题——读-写 / 写-读

➤ 为了避免在“读-写”情况下避免脏读、不可重复读、幻读现象，有如下两种可选的解决方案：

① 方案1：一致性读/一致性无锁读/快照读

读操作——MVCC；写操作——对记录进行加锁。

② 方案2：锁定读 (S锁/X锁)

读、写操作都采用加锁的方式。

由于前面已经介绍过MVCC了，那么下面我们主要针对锁定读和锁定写进行介绍

- 如果采用MVCC方式，读-写操作彼此并不冲突，性能更高；
- 如果采用加锁方式，读-写操作彼此需要排队执行，从而影响性能；
- 所有普通的SELECT语句在READ COMMITTED或REPEATABLE READ隔离级别下都算是一致性读。所以一般来说，我们会选择采用MVCC方式来解决读-写操作并发执行的问题，但是在某些特殊的业务场景（如：银行业务，需要读取最新数据和数据准确性，对执行时间并无苛刻要求），这时我们才会选择读、写都加锁的方式。

行级锁——锁定读操作

➤ 共享锁 / S锁 (Shared Lock)

在事务要读取一条记录时，需要先获取该记录的S锁。

SELECT ... LOCK IN SHARE MODE;

➤ 独占锁 / 排它锁 / X锁 (Exclusive Lock)

在事务要修改一条记录时，需要先获取该记录的X锁。

SELECT ... FOR UPDATE;

➤ S锁与X锁的兼容关系如下所示:

兼容性	X锁	S锁
X锁	不兼容	不兼容
S锁	不兼容	兼容

行级锁——锁定写操作

➤ 针对DELETE操作

先在B+树中定位到这条记录的位置，获取这条记录的X锁，最后再执行delete mark操作。

➤ 针对INSERT操作

一般情况下，新插入的一条记录受隐式锁保护，不需要在内存中为其生成对应的锁结构。

➤ 针对UPDATE操作，分为如下3种情况：

① 未修改主键并且被更新的列在修改前后所占用的存储空间未发生变化

先在B+树中定位到这条记录的位置，然后获取这条记录的X锁，最后在原记录的位置进行修改操作。

② 未修改主键并且被更新的列在修改前后所占用的存储空间发生变化

先在B+树中定位到这条记录的位置，然后获取这条记录的X锁，之后将原记录彻底删除掉（即：把记录彻底移入垃圾链表），最后再插入一条新记录。

③ 修改主键

相当于在原记录上执行DELETE操作之后再进行一次INSERT操作。加锁操作就需要按照DELETE和INSERT的规则进行了。

表级锁

➤ 表级共享锁 (S锁)

其他事务可以继续获得该表/该表中的某些记录的S锁。

其他事务不可以继续获得该表/该表中的某些记录的X锁。

➤ 表级独占锁 (X锁)

其他事务不可以继续获得该表/该表中的某些记录的X锁或S锁。

➤ 意向共享锁 (IS锁)

当事务准备在某条记录上加S锁时，首先需要在表级别加一个IS锁。

➤ 意向独占锁 (IX锁)

当事务准备在某条记录上加X锁时，首先需要在表级别加一个IX锁。

表级锁之间的关系

- IS锁和IX锁是表级锁，它们的提出仅仅为了在之后加表级别的S锁和X锁时可以快速判断表中的记录是否被上锁，以避免用遍历的方式来查看表中有没有上锁的记录；也就是说，IS锁和IX锁之间或者与自身都是彼此兼容的。
- 兼容性关系如下所示：

兼容性	X	IX	S	IS
X	不兼容	不兼容	不兼容	不兼容
IX	不兼容	兼容	不兼容	兼容
S	不兼容	不兼容	兼容	兼容
IS	不兼容	兼容	兼容	兼容

InnoDB 表级锁

- InnoDB存储引擎提供的表级S锁或者X锁相当“鸡肋”，只会有一些特殊情况下（比如系统崩溃恢复时）用到。
- IS锁和IX锁是表级锁，它们的提出仅仅为了在之后加表级别的S锁和X锁时可以快速判断表中的记录是否被上锁，以避免用遍历的方式来查看表中有没有上锁的记录。
- 当我们使用了auto_increment修饰列的时候，就会涉及到AUTO-INC锁和轻量级锁。其中：
innodb_autoinc_lock_mode系统变量，用来控制到底使用上述两种方式中的哪一种。
 - ① 0：一律采用AUTO-INC锁。
 - ② 1：混合使用。插入记录的数量确定时采用轻量级锁，不确定时采用AUTO-INC锁。
 - ③ 2：一律采用轻量级锁。

```
mysql> show global variables like 'innodb_autoinc_lock_mode';
+-----+-----+
| Variable_name | Value |
+-----+-----+
| innodb_autoinc_lock_mode | 2     |
+-----+-----+
1 row in set (0.00 sec)
```

InnoDB 行级锁——记录锁

➤ LOCK_REC_NOT_GAP

被称为**记录锁**，也就是仅仅负责把1条记录锁上的锁。

no	name	score
1	muse	90
3	bob	95
5	john	70
9	rose	65
13	james	88

给no值为5的记录加类型为LOCK_REC_NOT_GAP的记录锁

InnoDB 行级锁——gap 锁

➤ LOCK_GAP

被称为**gap锁**，锁住了指定记录**前面的间隙**，防止其间插入新记录。

gap锁的提出仅仅是为了**防止插入幻象记录**（即：幻读现象）而提出的。

no	name	score
1	muse	90
3	bob	95
5	john	70
9	rose	65
13	james	88

给no值为5的记录加类型为LOCK_GAP的记录锁

InnoDB 行级锁——next-key 锁

➤ LOCK_ORDINARY

被称为**next-key锁**，本质就是一个 **记录锁** + **gap锁** 的合体。它既能保护该条记录，又能阻止别的事务将新记录插入到被保护记录前面的间隙中。

no	name	score
1	muse	90
3	bob	95
5	john	70
9	rose	65
13	james	88

给no值为5的记录加类型为LOCK_ORDINARY的记录锁

InnoDB 行级锁——插入意向锁

➤ LOCK_INSERT_INTENTION

也被称为**插入意向锁**，事务在等待时也需要在内存中生成一个**锁结构**，表明有事务想在某个间隙中插入新记录，但是现在处于等待状态。

no	name	score
1	muse	90
3	bob	95
5	john	70
9	rose	65
13	james	88

锁结构		
trx信息	is_waiting	type
T1	false	gap

获取锁成功
为9加了gap锁
事务继续执行

锁结构		
trx信息	is_waiting	type
T2	true	插入意向锁

待插入no=7
获取锁失败
事务开始等待

锁结构		
trx信息	is_waiting	type
T3	true	插入意向锁

待插入no=6
获取锁失败
事务开始等待

InnoDB 行级锁——隐式锁 1

- 一般情况下，执行INSERT语句是不需要在内存中生成锁结构的。
- 但是也会有例外，比方说：一个事务（T1）首先插入了一条记录（此时并没有与该记录关联的锁结构），然后另一个事务（T2）执行如下操作：
 - ① 立即使用 `SELECT... LOCK IN SHARE MODE` 语句读取这条记录（也就是要获取这条记录的S锁），或者使用 `SELECT ... FOR UPDATE` 语句读取这条记录（也就是要获取这条记录的X锁），该咋办？如果允许这种情况的发生，那么可能出现脏读现象。
 - ② 立即修改这条记录（也就是要获取这条记录的X锁）该咋办？如果允许这种情况的发生，那么可能出现脏写现象。
- 解决办法——使用事务id，我们把聚簇索引和二级索引中的记录分开看一下：

InnoDB 行级锁——隐式锁 2

➤ 场景1: 对于**聚簇索引**

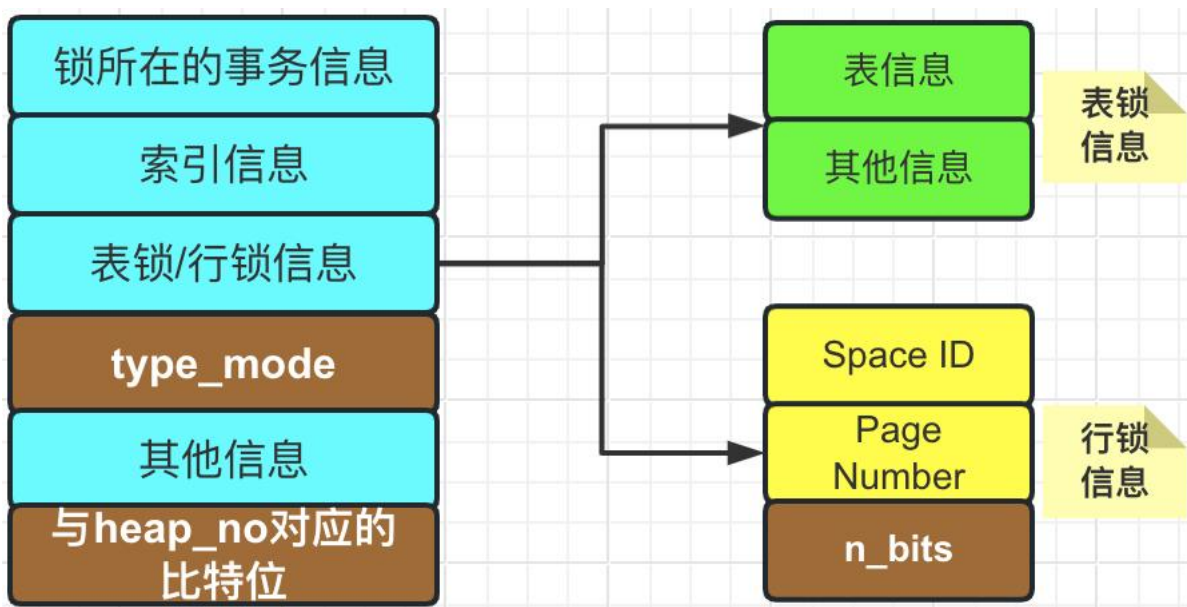
有一个**trx_id**隐藏列，该隐藏列记录着最后改动该记录的事务id。在当前事务中新插入一条聚簇索引记录后，该记录的**trx_id**隐藏列代表的就是当前事务的事务id。如果其他事务此时想对该记录添加S锁或者X锁，首先会看一下**该记录的trx_id隐藏列代表的事务是否是当前的活跃事务**。如果不是的话就可以正常读取；如果是的话，那么就帮助当前事务创建一个**X锁**的锁结构，该锁结构的**is_waiting**属性为**false**；然后为自己也创建一个锁结构，该锁结构的**is_waiting**属性为**true**，之后自己进入等待状态。

➤ 场景2: 对于**二级索引**

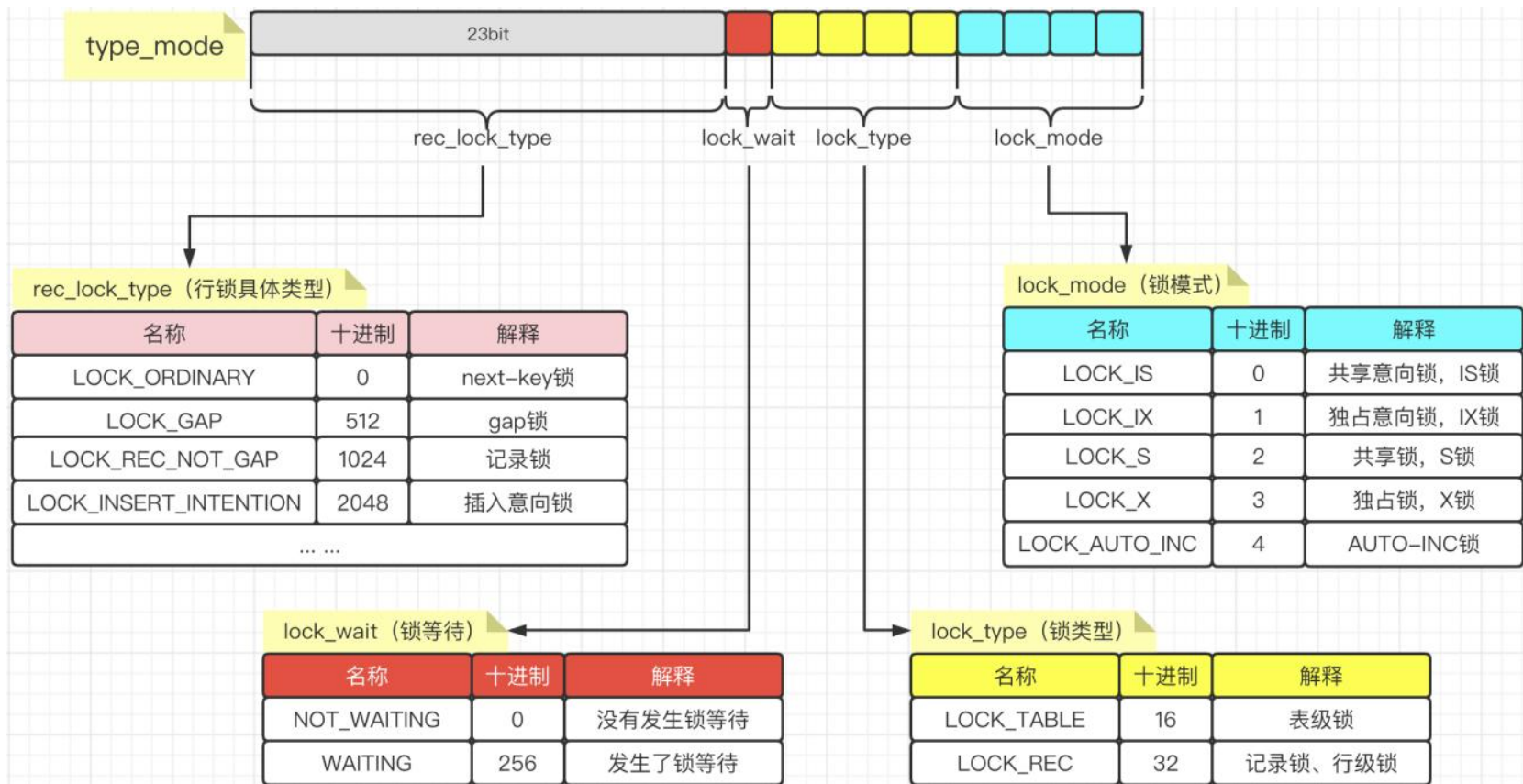
本身并没有**trx_id**隐藏列，但是在二级索引页面的Page Header 部分有一个**PAGE_MAX_TRX_ID**属性，该属性代表对该页面做改动的最大的事务id。如果**PAGE_MAX_TRX_ID**属性值**小于当前最小的活跃事务id**，那就说明对该页面做修改的事务都已经提交了，否则就需要在页面中定位到对应的二级索引记录，然后通过回表操作找到它对应的聚集索引记录，然后再重复情景1的做法。

InnoDB 锁的内存结构

- 前文已经介绍过，对一条记录加锁的本质就是在内存中创建一个锁结构跟这条记录相关联。那么，我们说了这么多次的锁结构，它到底是怎么组成的呢？它其实主要是由6部分组成的。分别为：锁所在的事务信息、索引信息、表锁或行锁信息、type_mode、其他信息、与heap_no对应的比特位。

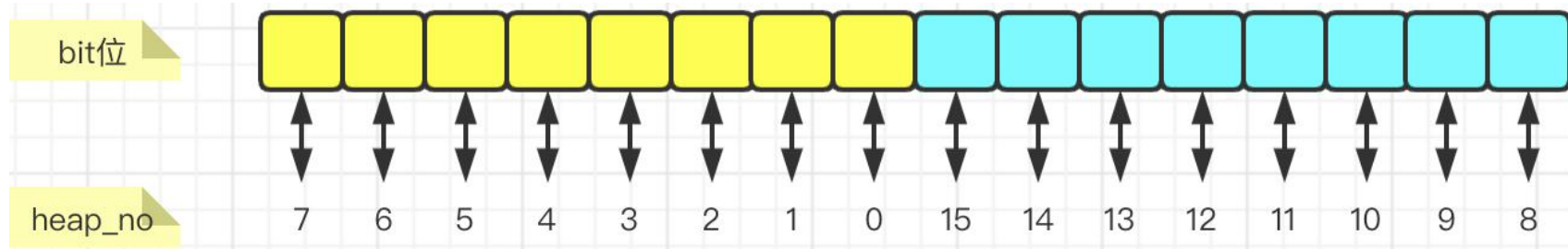


InnoDB锁的内存结构——type_mode



与 heap_no 对应的比特位

➤ 与 heap_no 对应的比特位 (以 bit=16 为例)

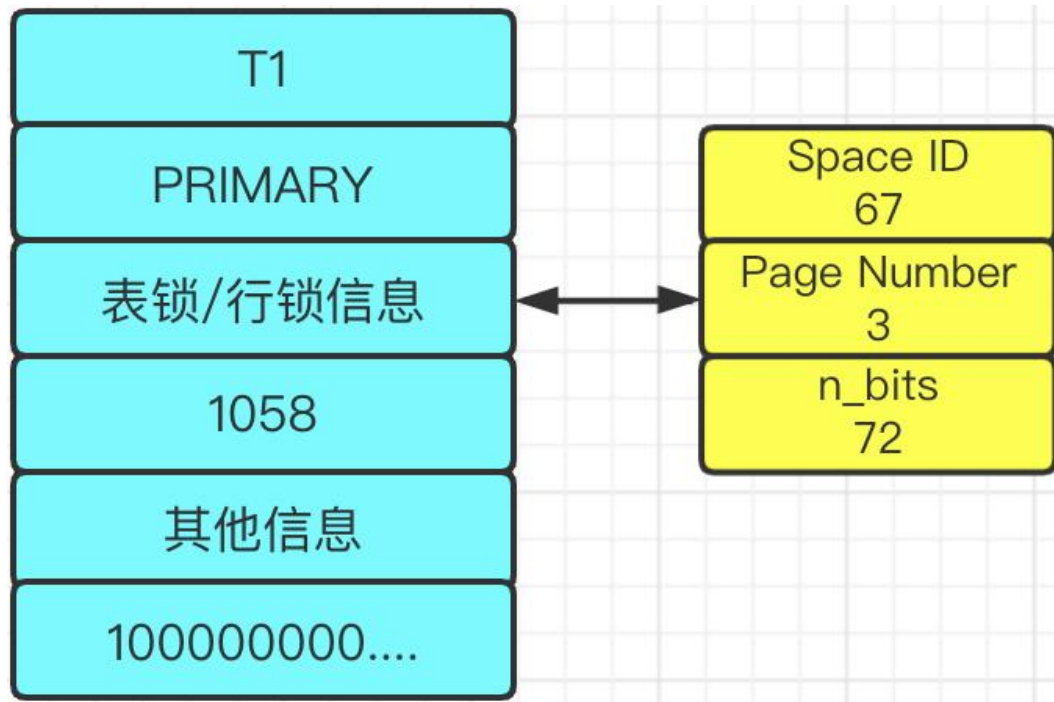


➤ n_bits 的计算公式如下所示:

$$n_bits = (1 + ((n_recs + LOCK_PAGE_BITMAP_MARGIN) / 8)) * 8$$

n_bit 计算公式	参数	解释
	n_recs	当前页面中一共有多少条记录
	LOCK_PAGE_BITMAP_MARGIN	默认值为64

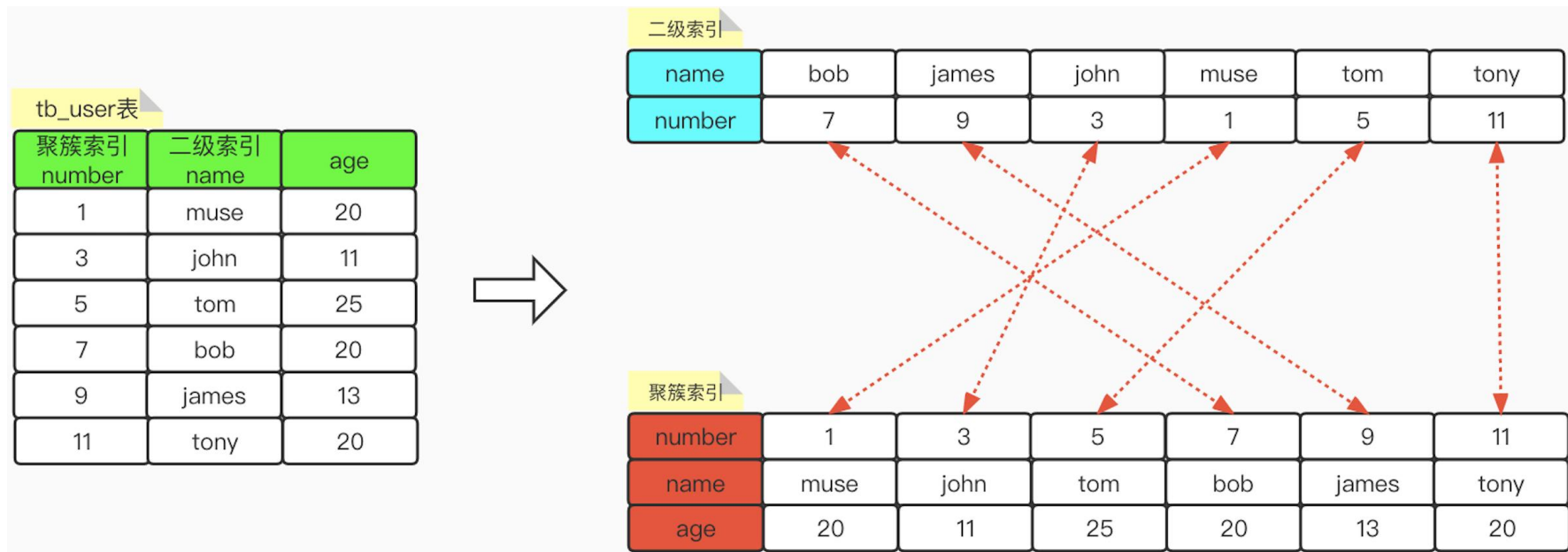
实践例子



➤ 实践例子:

我们假设要开启一个事务T1，往tb_user表（已存在5条记录）中表空间为67，页号为3的页面上，插入一个number=15的记录（number是主键），并为这个记录加S锁，那么我们分析一下它所生成的行级锁结构是怎样的？

加锁操作解析——前期准备工作



在tb_user表中，有3个字段，分别为：学号（number），学生姓名（name）和学生年龄（age）这3个字段。那么，其中number字段为聚簇索引，name字段为二级索引。

加锁操作解析——普通select语句

➤ 普通的select语句其实针对于**不同的隔离级别**，会有不同的处理方式。

隔离级别	加锁方式	存在的问题
READ UNCOMMITTED	不加锁，直接记录的最新版本	可能出现脏读、不可重复读和幻读
READ COMMITTED	不加锁，每次执行select时都会生成一个ReadView，可以避免脏读	可能出现不可重复读和幻读
REPEATABLE READ	不加锁，只在第一次执行select语句时生成一个ReadView	可以很大程度上解决幻读问题，但并不是完全解决
SERIALIZABLE	当autocommit=0时，select语句会被转换成select ... LOCK IN SHARED MODE，即：给记录加S锁。 当autocommit=1时，select语句不会加锁，只是利用MVCC生成一个ReadView来读取记录。因为启动了自动提交，意味着一个事务中只包含一条语句，那么只执行一条语句，也就不会出现重复读和幻读了。	不会出现脏读、不可重复读和幻读

加锁实战分析

➤ 那么针对与语句加锁分析，我们根据以下4类进行分析，分别为：

【1】普通的SELECT语句

【2】锁定读的语句

【3】半一致性读的语句

【4】INSERT语句

加锁操作解析——锁定读语句

- 对锁定读的语句，其实可以归类为以下四种语句：

语句1: `SELECT ... LOCK IN SHARE MODE;`

语句2: `SELECT ... FOR UPDATE;`

语句3: `UPDATE ...`

语句4: `DELETE ...`

- 不同的隔离级别，为当前记录加不同类型的锁。在不满足查询条件的情况下，如果隔离级别为`READ UNCOMMITTED`或`READ COMMITTED`，则要释放掉加在该记录上面的锁；如果隔离级别为`REPEATABLE READ`或`SERIALIZABLE`，则不去释放记录上面的锁。

隔离级别	加锁类型
READ UNCOMMITTED	记录锁
READ COMMITTED	记录锁
REPEATABLE READ	next-key锁
SERIALIZABLE	next-key锁

SELECT ... LOCK IN SHARE MODE 示例一

- 针对**聚簇索引**number作为搜索条件，隔离级别为**READ UNCOMMITTED**或**READ COMMITTED**，执行**select * from tb_user where number >2 AND number <=7 AND age=25 LOCK IN SHARE MODE**;

聚簇索引		1	2	3	4	
number	1	3	5	7	9	11
name	muse	john	tom	bob	james	tony
age	20	11	25	20	13	20

不符合 age=25, 所以先加锁, 后释放锁

S记录锁

不符合 age=25, 所以先加锁, 后释放锁

超出了 (2,7] 的边界, 所以先加锁, 后释放锁

【索引类型】

聚簇索引

【隔离级别】

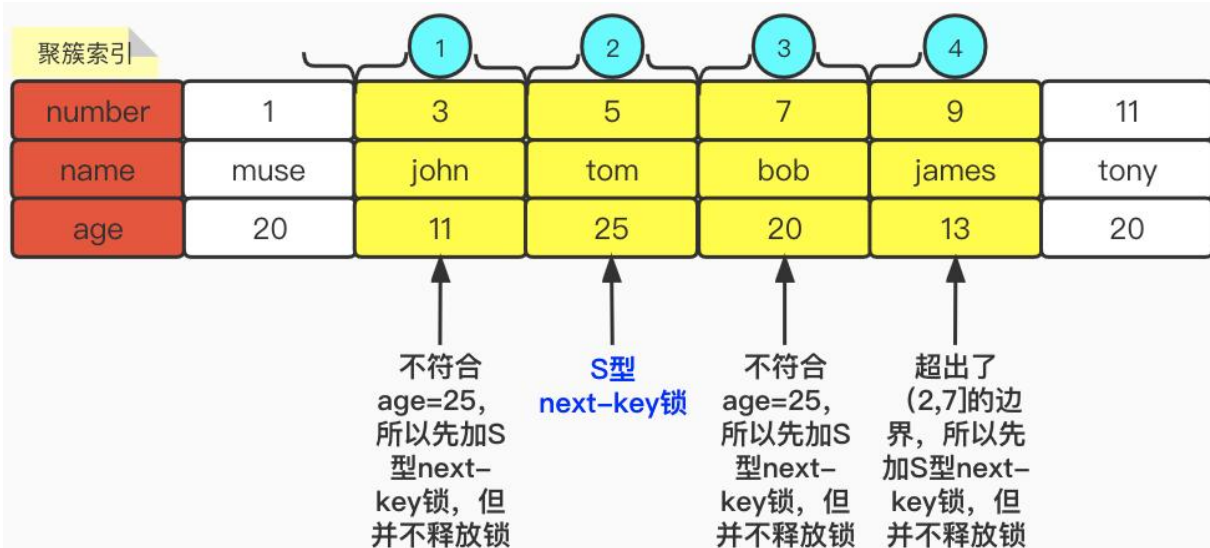
READ UNCOMMITTED

READ COMMITTED

select * from
tb_user
where
number >2 AND
number <=7 AND
age=25
LOCK IN SHARE MODE;

SELECT ... LOCK IN SHARE MODE 示例二

- 针对**聚簇索引**number作为搜索条件，隔离级别为**REPEATABLE READ**或**SERIALIZABLE**，执行 `select * from tb_user where number >2 AND number <=7 AND age=25 LOCK IN SHARE MODE;`



【索引类型】

聚簇索引

【隔离级别】

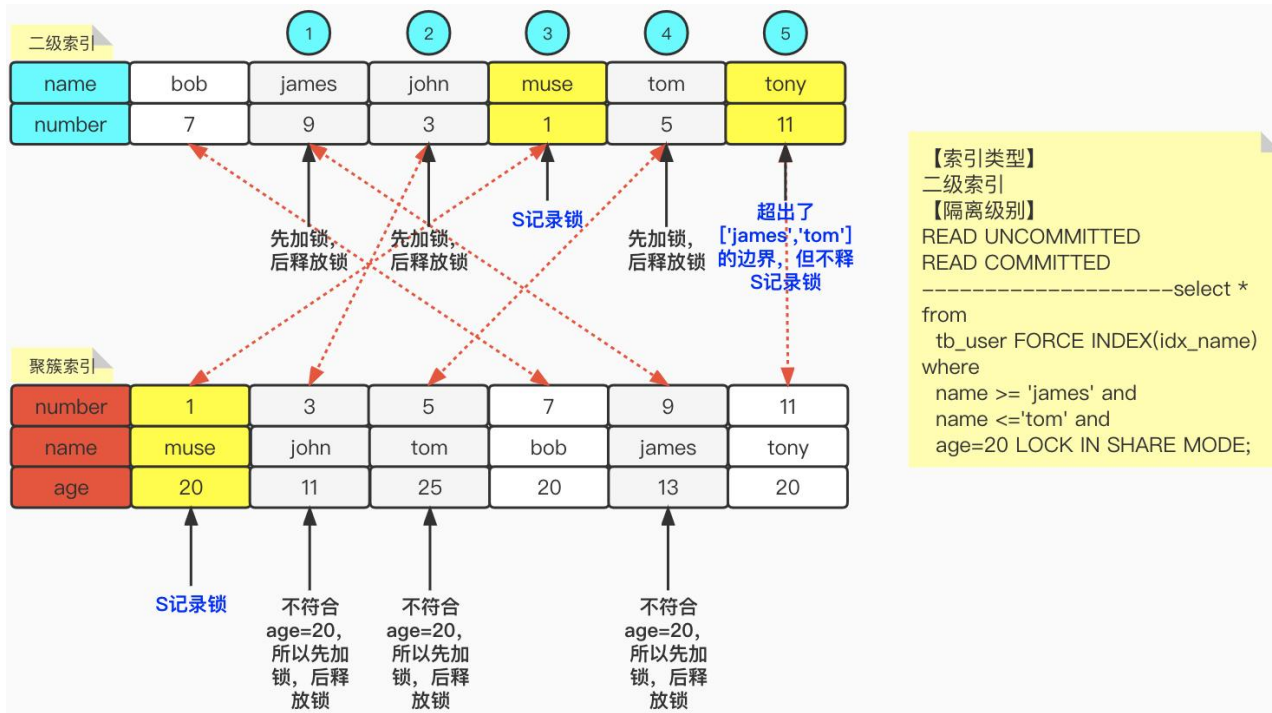
REPEATABLE READ

SERIALIZABLE

```
select * from
  tb_user
where
  number >2 AND
  number <=7 AND
  age=25
LOCK IN SHARE MODE;
```

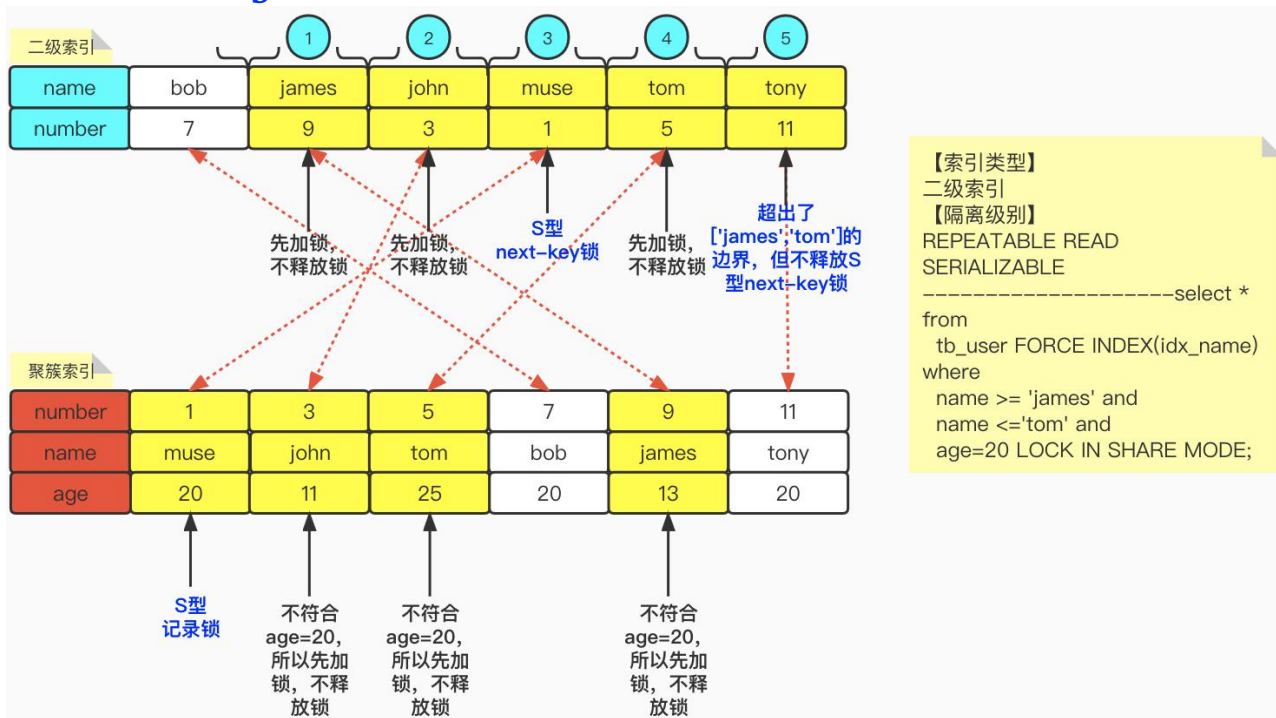
SELECT ... LOCK IN SHARE MODE 示例三

- 针对**二级索引**name作为搜索条件，隔离级别为**READ UNCOMMITTED**或**READ COMMITTED**，执行`select * from tb_user FORCE INDEX(idx_name) where name >= 'james' AND name <='tom' AND age=20 LOCK IN SHARE MODE;`



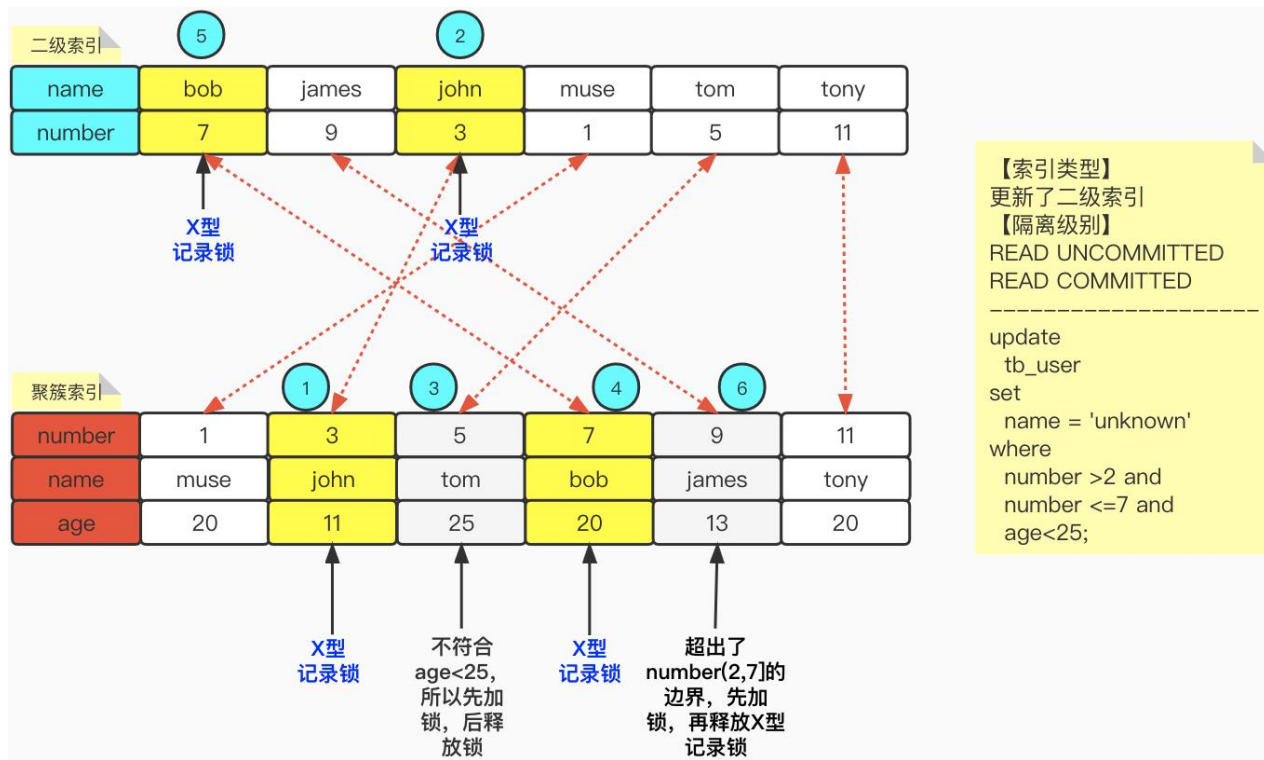
SELECT ... LOCK IN SHARE MODE 示例四

- 针对**二级索引**name作为搜索条件，隔离级别为**REPEATABLE READ**或**SERIALIZABLE**，执行
`select * from tb_user FORCE INDEX(idx_name) where name >= 'james' AND
name <='tom' AND age=20 LOCK IN SHARE MODE;`



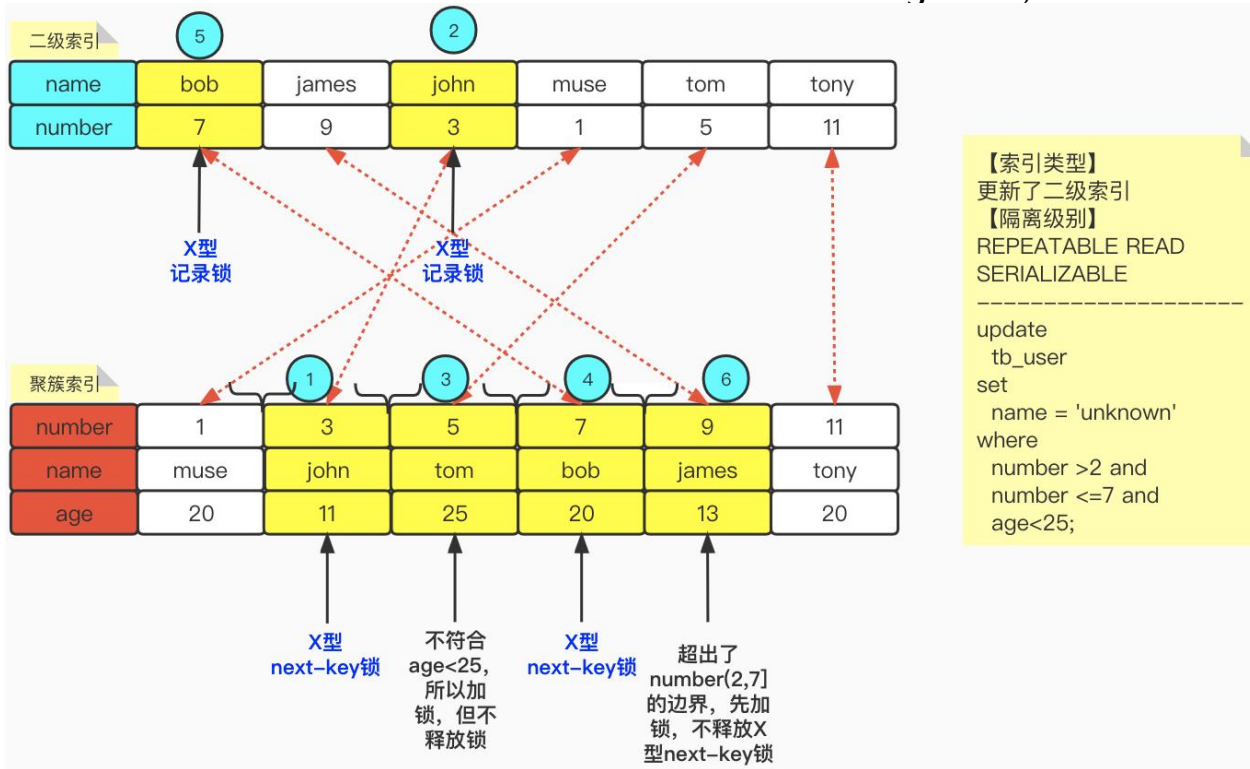
UPDATE ... 示例一

- 隔离级别为**READ UNCOMMITTED**或**READ COMMITTED**，执行`update tb_user set name = 'unknown' where number >2 AND number <=7 AND age<25;`



UPDATE ... 示例二

- 隔离级别为 **REPEATABLE READ** 或 **SERIALIZABLE**, 执行 `update tb_user set name = 'unknown' where number >2 AND number <=7 AND age<25;`

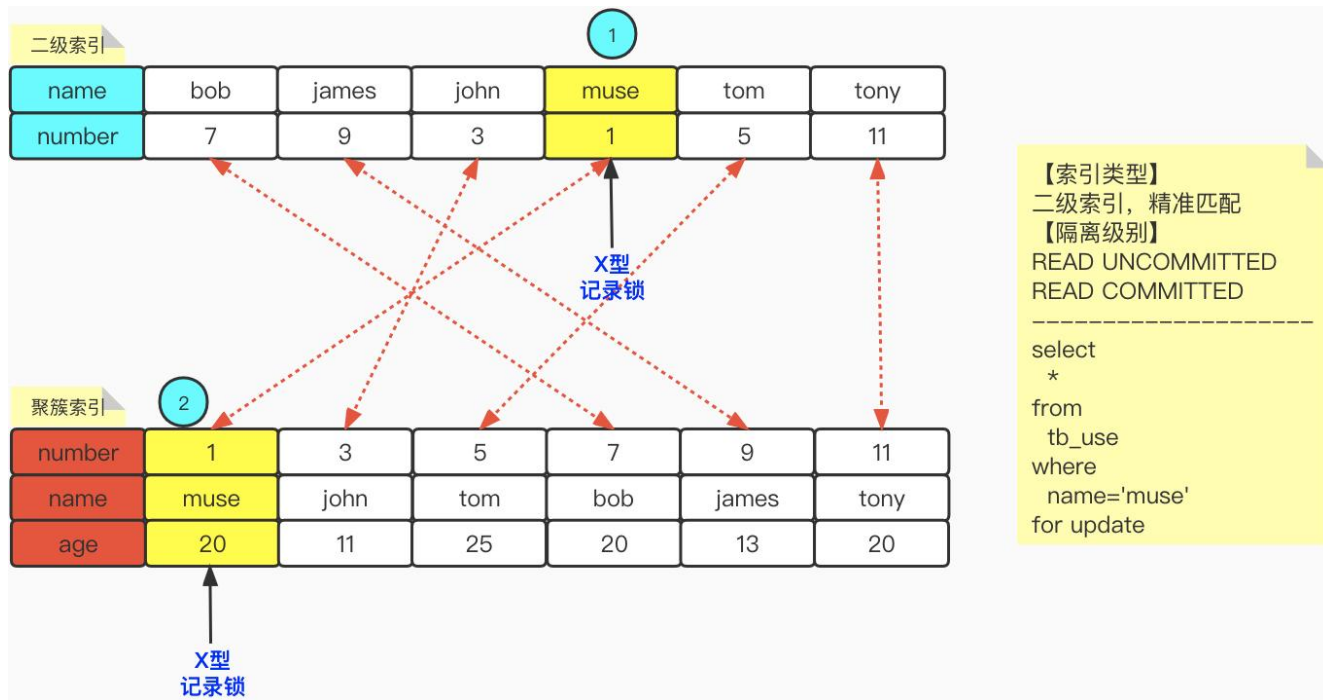


SELECT ... FOR UPDATE 示例 & DELETE ... 示例

- SELECT ... FOR UPDATE语句的加锁过程与SELECT ... LOCK IN SHARE MODE语句类似，区别是为记录加X锁。
- DELETE ... 语句的加锁过程与UPDATE的处理方式相同，当表中包含二级索引，那么二级索引记录在被删除之前都需要加X型记录锁。
- 对于UPDATE和DELETE语句来说，在对被更新或者被删除的二级索引记录加锁的时候，实际加的是隐式锁，但是效果与X型记录锁一样。
- 对于隔离级别为READ UNCOMMITTED和READ COMMITTED的情况，采用的是一种称为半一致读的方式来执行UPDATE语句。

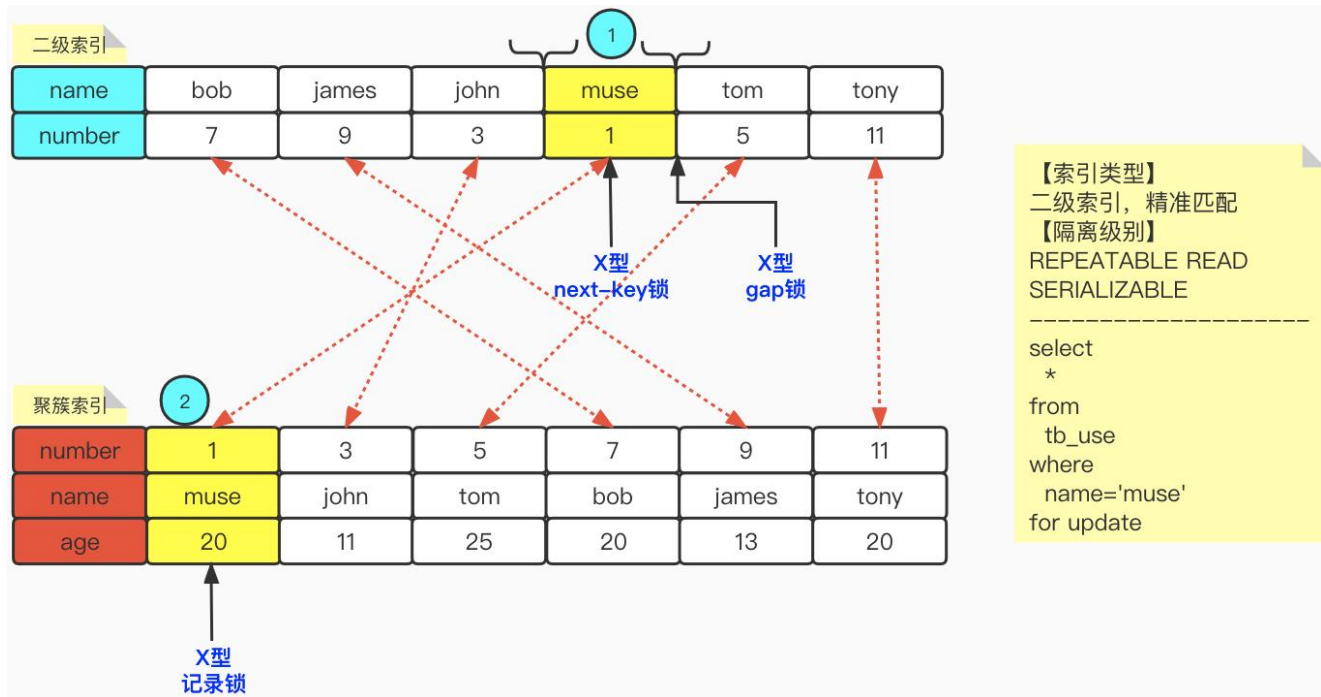
补充内容：二级索引精准匹配加锁流程（1）

- 当隔离级别为 **READ UNCOMMITTED** 和 **READ COMMITTED** 的情况，如果匹配的模式为 **精准匹配**，那么 **将不会为扫描区间后面的下一条记录加锁**。比如我们执行 `select * from tb_use where name='muse' for update`。那么加锁情况如下所示：



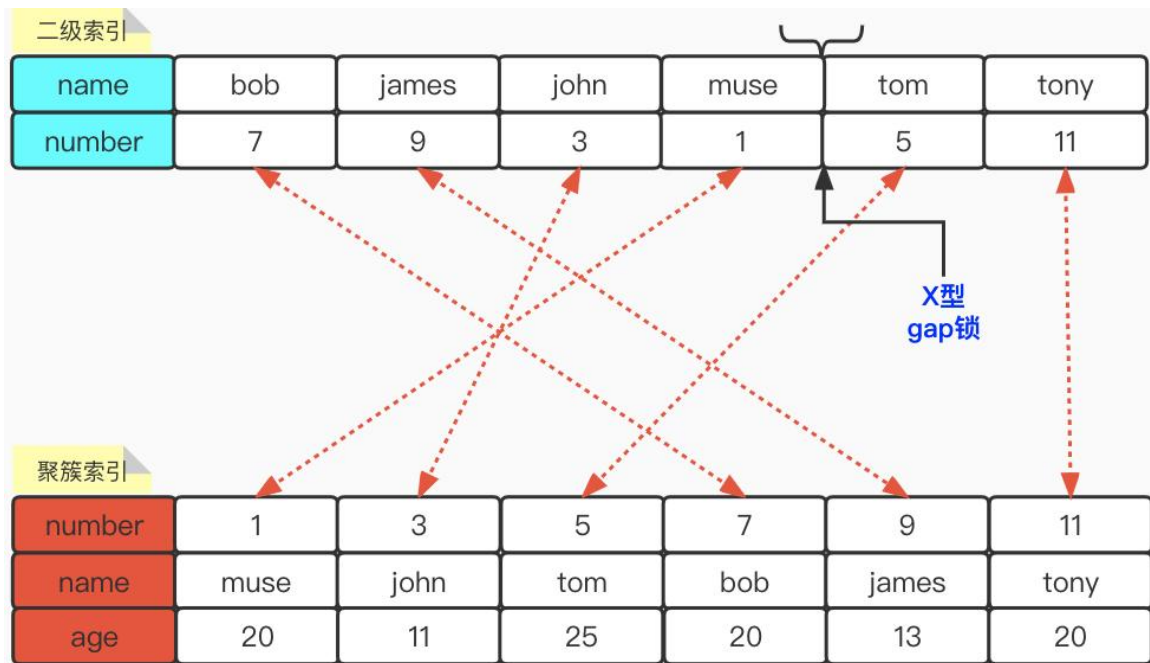
补充内容：二级索引精准匹配加锁流程（2）

- 当隔离级别为 **REPEATABLE READ** 或 **SERIALIZABLE** 的情况，如果匹配的模式为 **精准匹配**，那么会为扫描区间后面的下一条记录加 **gap锁**。比如我们执行 `select * from tb_use where name='muse' for update`。那么加锁情况如下所示：



补充内容：二级索引找不到记录（1）

- 当扫描区间中**没有记录**，且为**精确查找**，隔离级别为**REPEATABLE READ**或**SERIALIZABLE**，那么也要为扫描区间**后面的下一条记录加一个gap锁**。比如执行：`select * from tb_user where name='moon' FOR UPDATE`。如下所示：



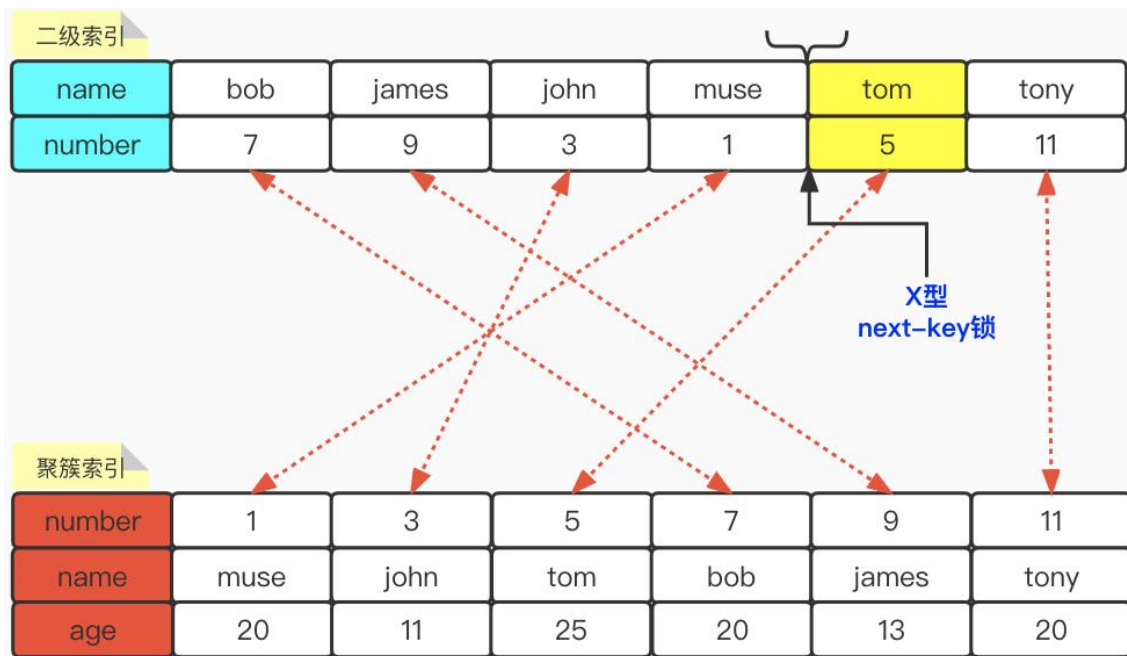
【索引类型】
二级索引，精准匹配，查询不到记录

【隔离级别】
REPEATABLE READ
SERIALIZABLE

```
select
*
from
  tb_use
where
  name='moon'
for update
```

补充内容：二级索引找不到记录（2）

- 当扫描区间中**没有记录**，且**不是精确查找**，隔离级别为**REPEATABLE READ**或**SERIALIZABLE**，那么也要**为扫描区间后面的下一条记录加一个next-key锁**。比如执行：
`select * from tb_user where name>'m' and name<'t' FOR UPDATE`。如下所示：

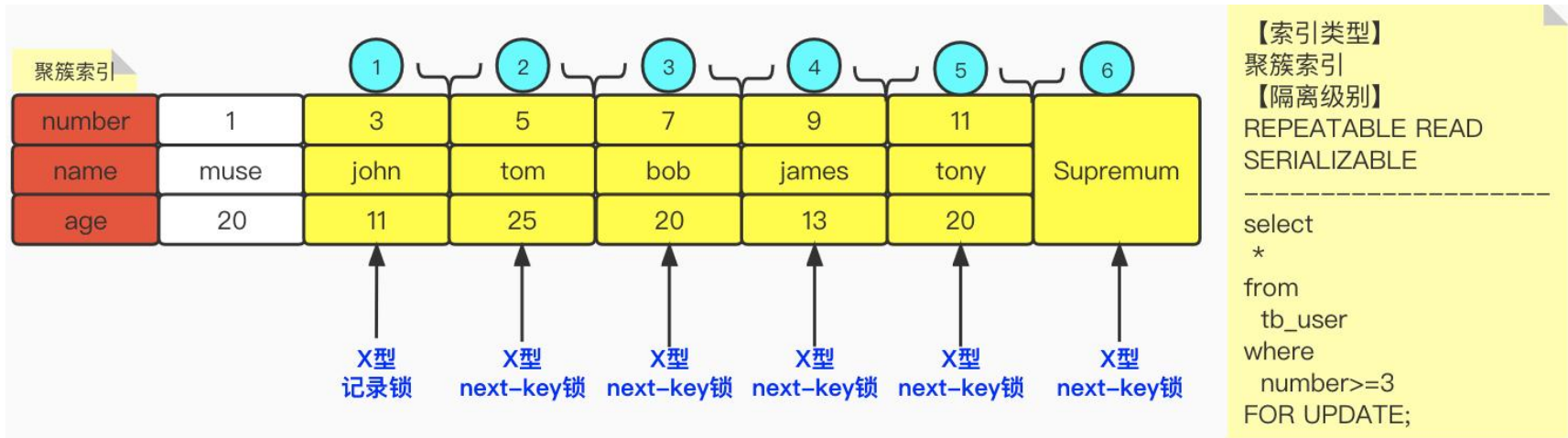


【索引类型】
二级索引，不是精准匹配，
查询不到记录
【隔离级别】
REPEATABLE READ
SERIALIZABLE

```
select
*
from
tb_use
where
name> 'm' and
name < 't'
for update
```

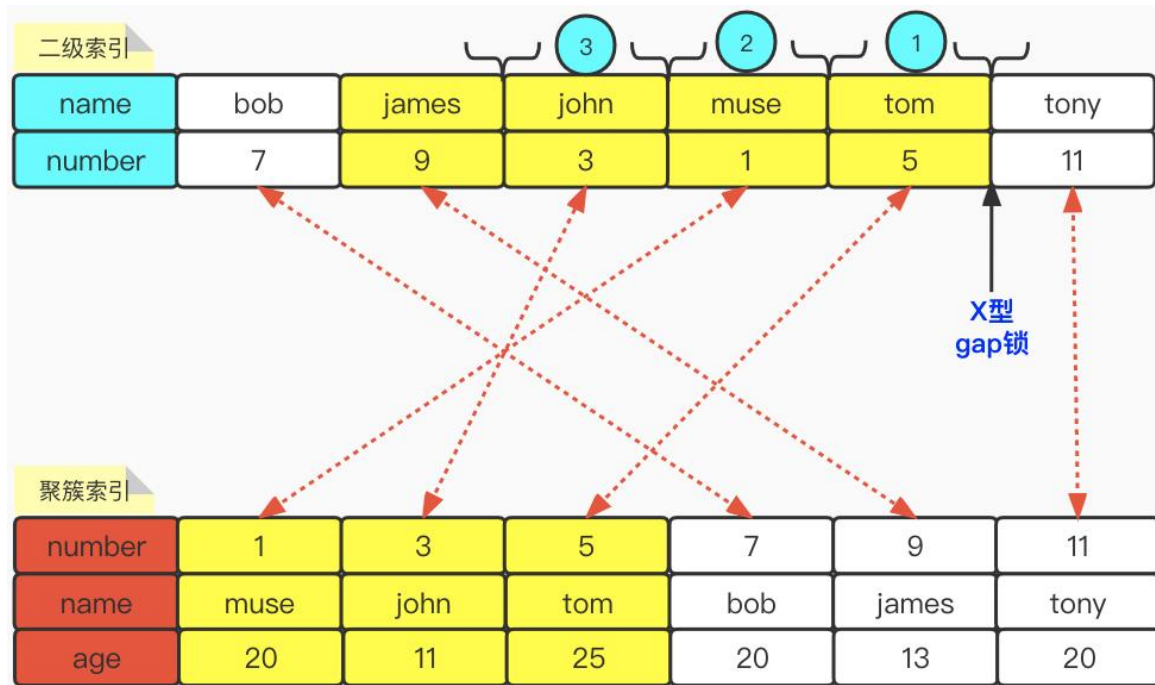
补充内容：左闭区间加锁

- 当隔离级别为 **REPEATABLE READ** 或 **SERIALIZABLE**，使用 **聚簇索引**，并且扫描区间为 **左闭区间**，如果定位到的第一个聚簇索引记录的 **number** 值正好与扫描区间中最小的值 **相同**，那么会为该聚簇索引记录加 **X类型** 的记录锁。例如：`select * from tb_user where number >= 3 FOR UPDATE;` 加锁情况如下所示：



补充内容：自右向左扫描加锁

- 当隔离级别为 **REPEATABLE READ** 或 **SERIALIZABLE**，从 **右向左** 的顺序扫描记录，会 **给匹配到的第一条记录的下一条记录加gap锁**。例如：select * from tb_user where name >='john' and name <=tom and age=20 order by name **DESC** FOR UPDATE;



【索引类型】

二级索引

【隔离级别】

REPEATABLE READ

SERIALIZABLE

select

*

from

tb_user

where

name >='john' and

name <=tom and

age=20

order by name

DESC FOR UPDATE;

加锁操作解析——半一致性读的语句

- 当隔离级别为READ UNCOMMITTED或READ COMMITTED且执行UPDATE语句时，将会使用半一致读。
- 什么是半一致读呢？

就是当UPDATE语句读取到已经被其他事务加了X锁的记录时，InnoDB会将该记录的最新提交版本读出来，然后判断该版本是否与UPDATE语句中的搜索条件相匹配。如果不匹配，则不对该记录加锁，从而跳到下一条记录；如果匹配，则再次读取该记录并对其进行加锁。这样做的目的就是让UPDATE语句尽量少被别的语句阻塞。

加锁操作解析——INSERT语句

- insert语句在一般情况下不需要在内存中生成锁结构，只是单纯依靠隐式锁保护插入的记录。
- 不过在当前事务插入一条记录之前，需要先定位该记录在B+树中的位置。如果该位置的下一条记录已经被加了gap锁或next-key锁。那么，当前事务就会为该记录加上插入意向锁，并且事务进入等待状态。

End



微信搜一搜



爪哇繆斯